DBMS Practical Task

Customer Database Management and Error Detection Using MySQL

Name	Shaik Manjoor Ahmed
Student ID	24SDC0407
Course	BCA - B Section (2024-2027)
Tool	MySQL 8.0 / MySQL Workbench
Date	17 October 2026

1. Objective

Create a customer database of 500 synthetic records, find the data errors in it, repair what can be repaired, quarantine what cannot, and add constraints so the same errors cannot enter again. All data is synthetic.

2. Before and after summary

Records checked	Errors found	Corrected in place	Removed	Valid remaining
500	50	12	38	462

Customers_Before keeps the original 500 rows. Customers_Rejected keeps the 38 removed rows with the reason, so nothing is lost silently.

3. Errors found and how each was handled

Error type	Found	How it was detected	Action and reason
Duplicate customer IDs	5	GROUP BY Customer_ID HAVING COUNT(*) > 1	Corrected: first row kept, later rows given a new unique ID. These were different people, so no record is deleted.
Duplicate customer records	10	GROUP BY LOWER(TRIM(Email)), count > 1	Removed: earliest copy kept, the rest moved to Customers_Rejected.
Missing names	7	Name IS NULL or empty after TRIM	Removed: a name cannot be guessed.
Missing emails	6	Email IS NULL or empty after TRIM	Removed: an email cannot be guessed.
Invalid emails	7	NOT REGEXP email pattern	4 corrected (stray spaces, double @). 3 removed (missing @, missing .com, empty domain).
Incorrect phone numbers	10	Not 10 digits starting with 6-9	3 corrected (+91 prefix, dash, space). 7 removed (too short, letters, extra digit, invalid start).
Invalid ages	5	NULL or not between 18 and 100	Removed: true age is unknown.

Rule used: a record is repaired only when the correct value is certain (for example, removing a space). If the true value is unknown, guessing would create new bad data, so the record is quarantined instead.

4. What the regular expressions check

Email: letters/digits/symbols, then exactly one @, then a domain name, then a dot, then an ending of 2 or more letters.
Phone: exactly 10 digits and the first digit must be 6, 7, 8 or 9 (Indian mobile format).

5. Constraints added

Constraint	Column	Blocks
PRIMARY KEY	Customer_ID	Duplicate IDs (error 1062)
UNIQUE	Email	Duplicate emails (error 1062)
NOT NULL	Customer_Name, Email	Missing name or email (error 1048)
CHECK chk_phone	Phone_Number	Phone not matching 10 digits starting 6-9 (error 3819)
CHECK chk_age	Age	Age outside 18-100 (error 3819)

A PRIMARY KEY stops duplicate IDs only. The same person can still be entered with a different ID, so duplicate names, emails or phones still need detection queries (and the UNIQUE key on email).

5A. Table structure (data dictionary)

Column	Type	Purpose	Rule after cleaning
Row_No	INT AUTO_INCREMENT	Helper key used only during cleaning	Dropped in Step 9
Customer_ID	INT	Customer identifier	PRIMARY KEY
Customer_Name	VARCHAR(100)	Full name	NOT NULL
Email	VARCHAR(150)	Contact email	NOT NULL, UNIQUE, valid format
Phone_Number	VARCHAR(20)	Mobile number	CHECK: 10 digits, starts with 6-9
Age	INT	Customer age	CHECK: between 18 and 100
City	VARCHAR(50)	City of the customer	No rule
Registration_Date	DATE	Date of registration	No rule

The Customers table is first created without any constraints, on purpose, so that bad data can be loaded and then detected. Customers_Before is a copy of all 500 original rows. Customers_Rejected has the same columns plus Reject_Reason.

5B. Method followed

1. **Create and load:** create the database and the raw table, then insert 500 synthetic rows with a recursive query.
2. **Inject errors:** run the given UPDATE statements to add 50 known errors, so there is something to find.
3. **Detect:** run one query per error type and compare the count with the expected count.
4. **Back up:** copy the 500 rows to Customers_Before and create Customers_Rejected before changing anything.
5. **Fix:** repair only when the right value is certain. Otherwise copy the row to Customers_Rejected (with the reason) and then delete it.
6. **Protect:** add PRIMARY KEY, UNIQUE, NOT NULL and CHECK constraints.
7. **Prove:** produce the before/after report, run the clean-check queries (0 rows), and show that bad inserts are rejected.

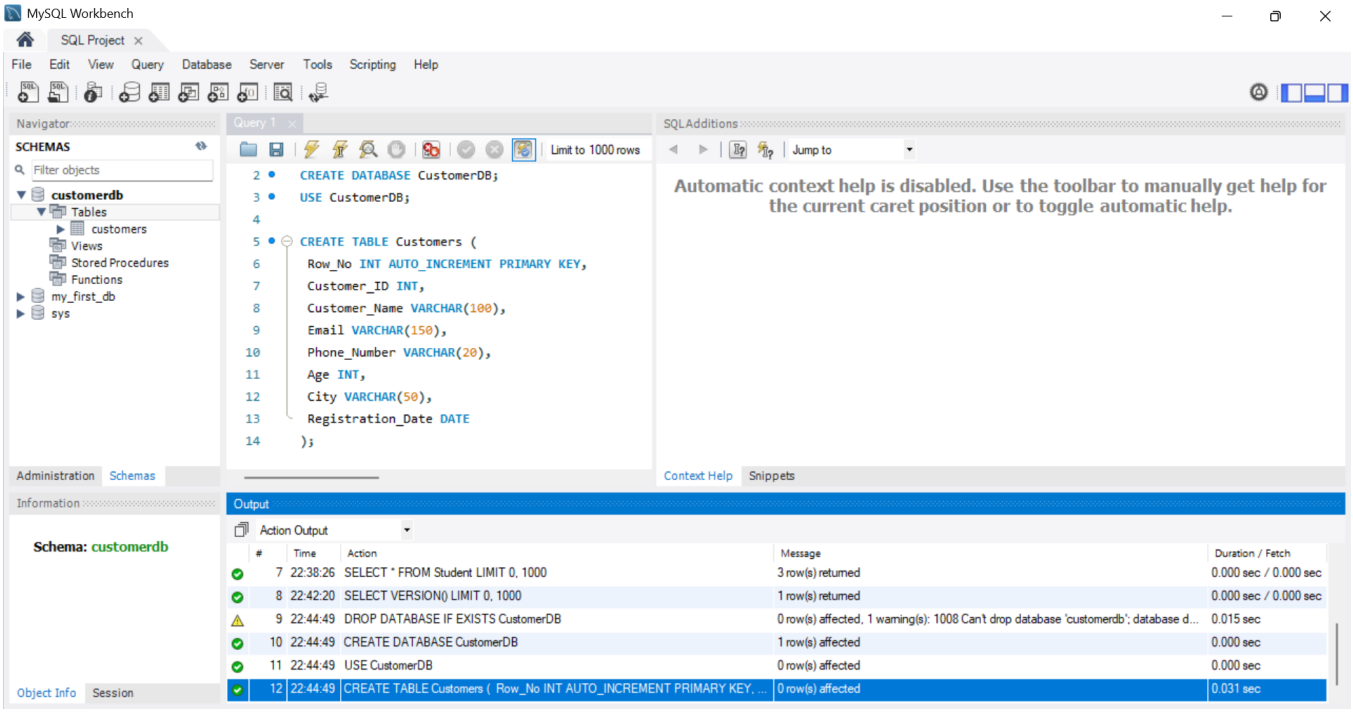
6. Problems faced during execution

Error 1055 on query 6.2. Selecting Email while grouping by LOWER(TRIM(Email)) failed because of the ONLY_FULL_GROUP_BY mode. Fixed by removing that mode for the session with SET SESSION sql_mode, after which the query returned the expected 10 groups.

Duplicate-ID counter showed 0. The first run of the report showed 45 errors / 7 corrected instead of 50 / 12, because the variable @fix_dup_id held 0. I confirmed that 5 rows had been repaired (IDs 1491 to 1495, screenshot in Step 8) and set @fix_dup_id = 5, then re-ran the report. It now shows 50 and 12.

7. Screenshots of execution

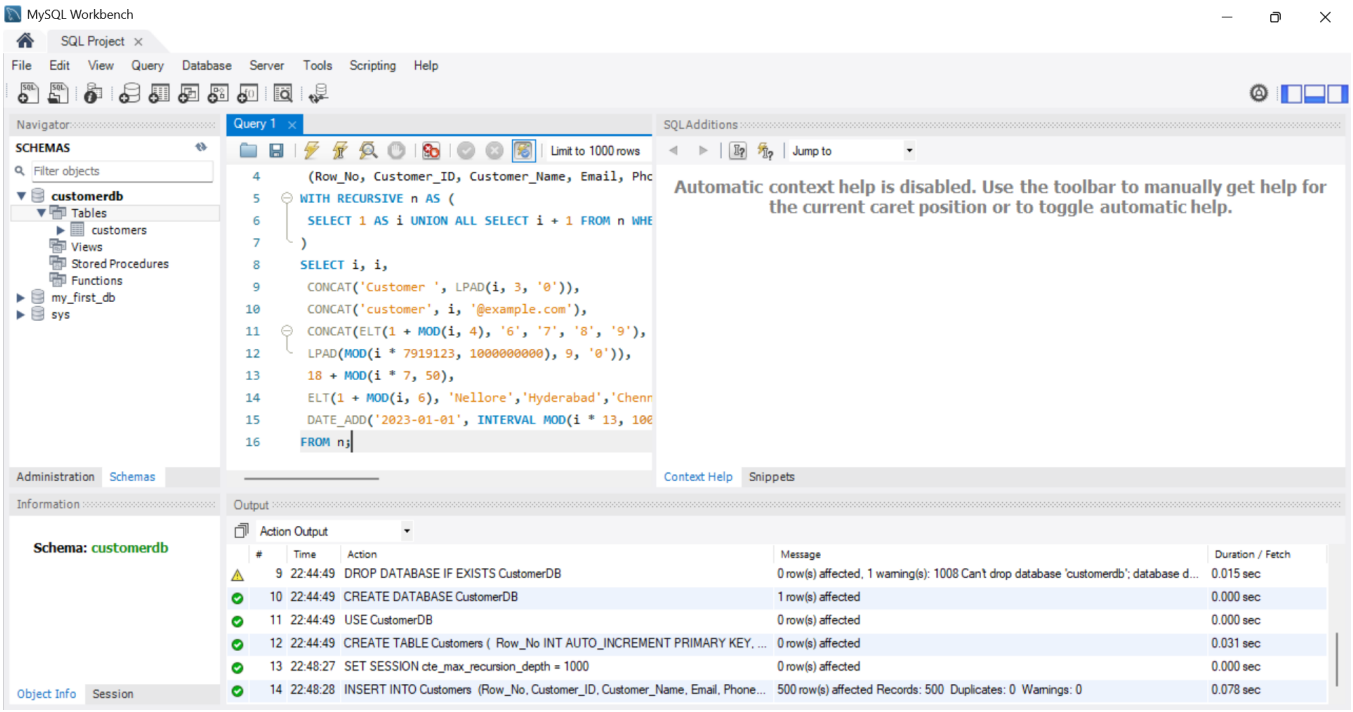
Step 1 and 2: Create database CustomerDB and the Customers table



The database and the raw Customers table (8 columns, no constraints) were created. The Action Output shows all three statements succeeded. The table is deliberately unconstrained so bad data can be loaded.

```
DROP DATABASE IF EXISTS CustomerDB;
CREATE DATABASE CustomerDB;
USE CustomerDB;
CREATE TABLE Customers (Row_No INT AUTO_INCREMENT PRIMARY KEY, Customer_ID INT,
  Customer_Name VARCHAR(100), Email VARCHAR(150), Phone_Number VARCHAR(20),
  Age INT, City VARCHAR(50), Registration_Date DATE);
```

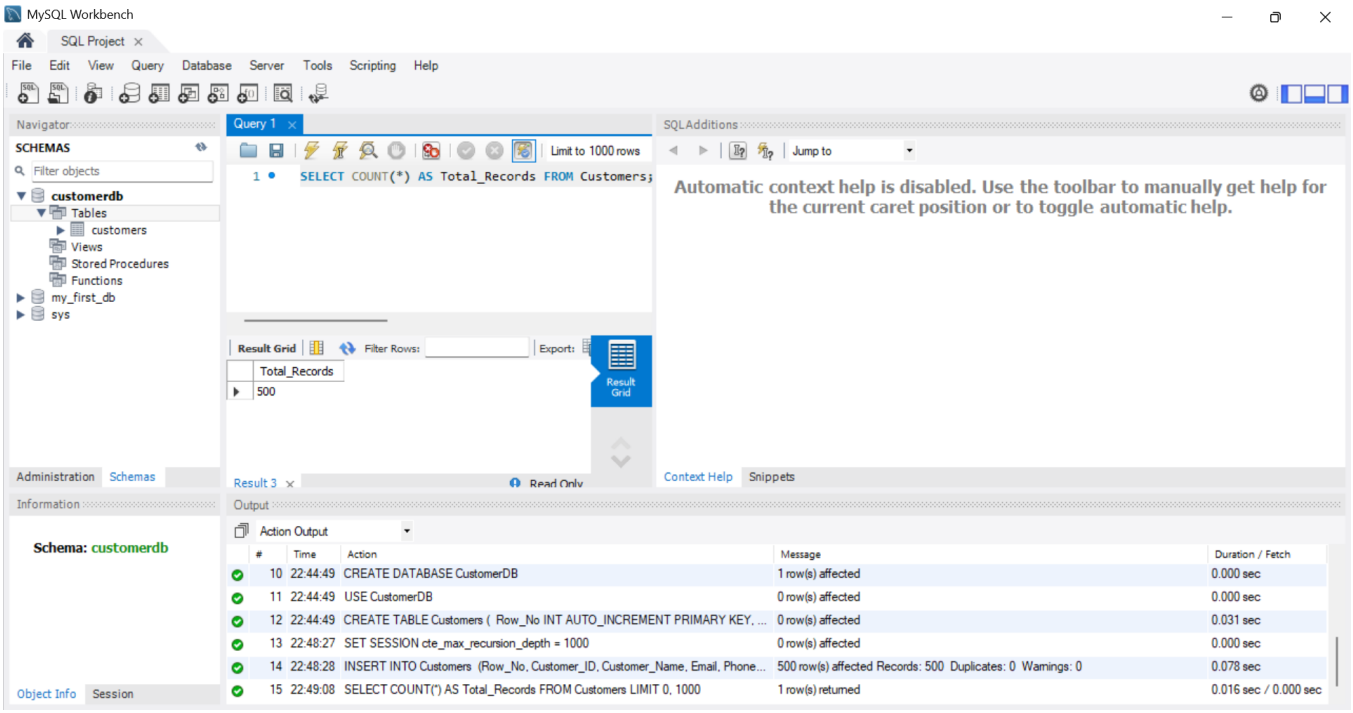
Step 3: Insert 500 synthetic records



A recursive query generated 500 rows. Action Output shows 500 row(s) affected, 0 duplicates, 0 warnings.

```
SET SESSION cte_max_recursion_depth = 1000;
INSERT INTO Customers (Row_No, Customer_ID, Customer_Name, Email, Phone_Number,
  Age, City, Registration_Date)
WITH RECURSIVE n AS (SELECT 1 AS i UNION ALL SELECT i + 1 FROM n WHERE i < 500)
SELECT i, i, CONCAT('Customer ', LPAD(i,3,'0')), ... FROM n;
```

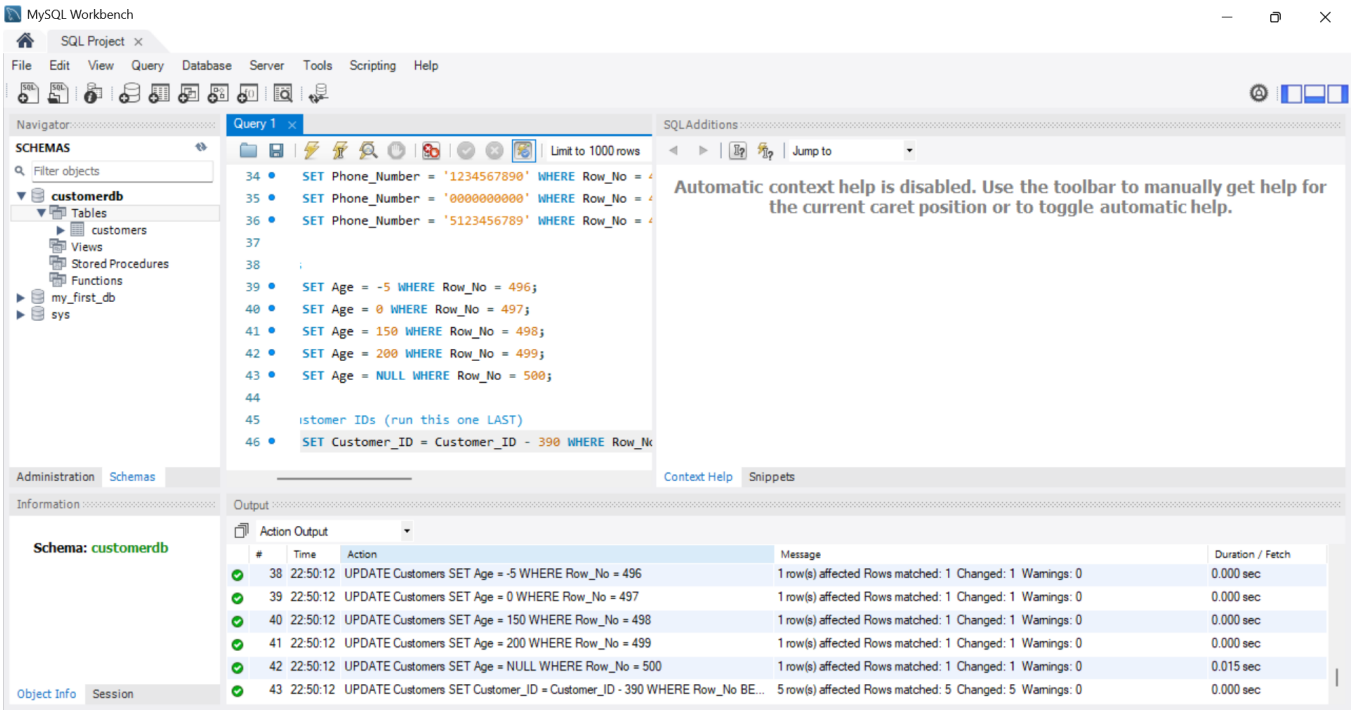
Step 5: Total record count



SELECT COUNT(*) returns 500, so all records are loaded.

```
SELECT COUNT(*) AS Total_Records FROM Customers;
```

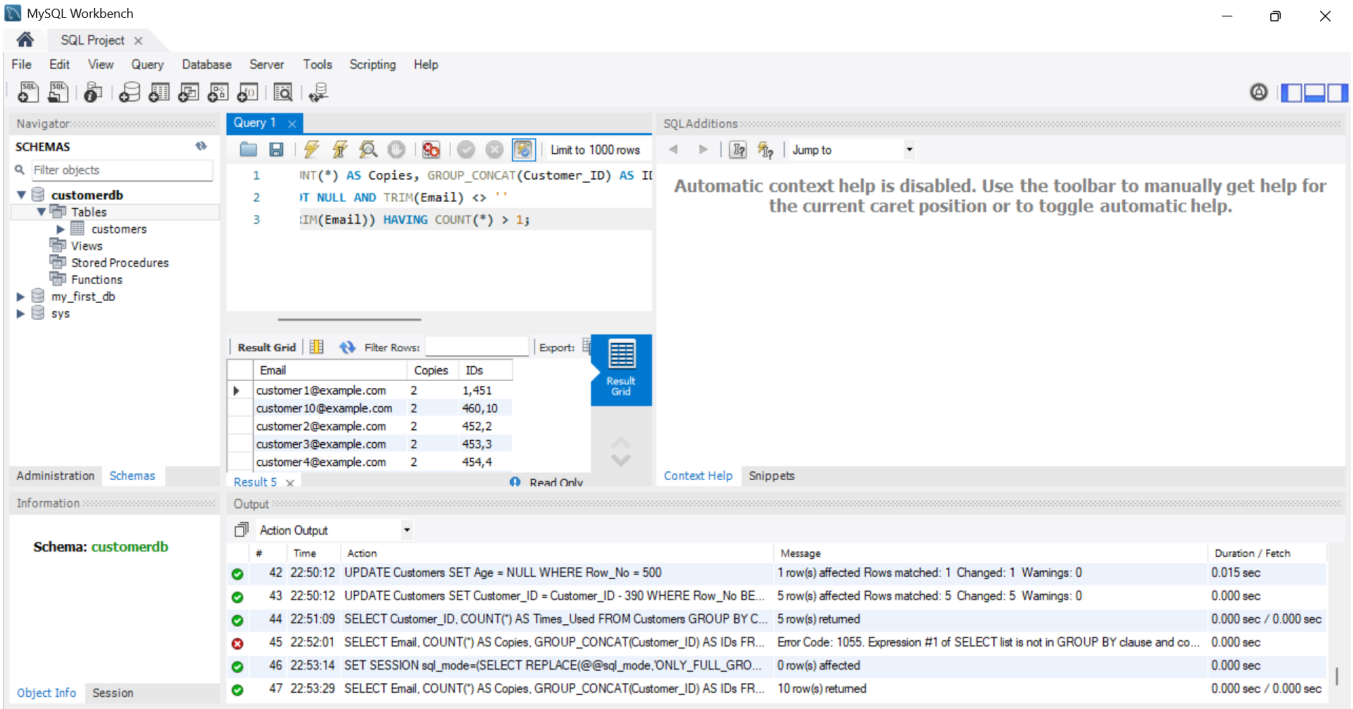
Step 4: Inject the 50 known errors



UPDATE statements added the errors (duplicate records, missing and invalid values). The last statement, which creates the 5 duplicate IDs, affected 5 rows.

```
UPDATE Customers SET Age = -5 WHERE Row_No = 496;   -- and the other injected errors
UPDATE Customers SET Customer_ID = Customer_ID - 390 WHERE Row_No BETWEEN 491 AND 495;
```

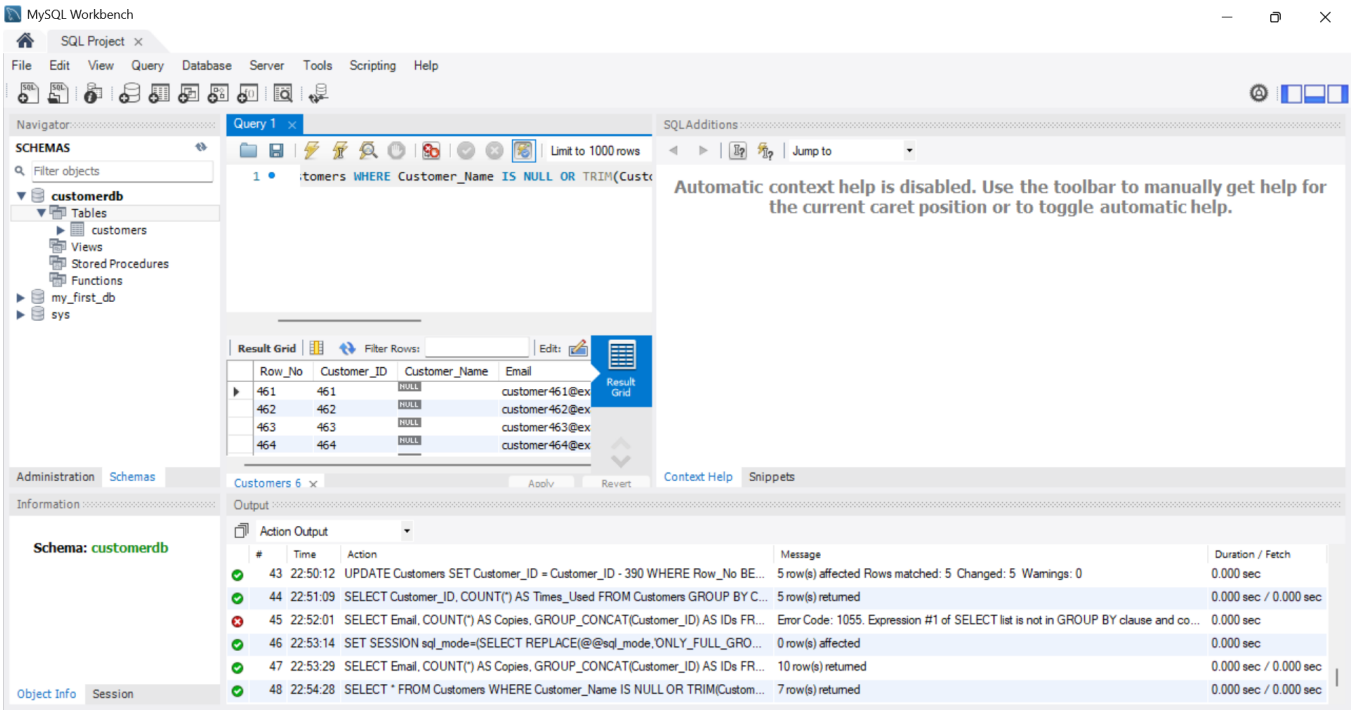
Step 6.1 and 6.2: Duplicate IDs and duplicate records



Query 6.1 (Action Output row 44) returned 5 duplicated IDs. Query 6.2 returned 10 email groups that appear twice, which are the duplicate customer records. The red entry above it is the error 1055 explained in section 6, followed by the corrected run.

```
SELECT Customer_ID, COUNT(*) AS Times_Used FROM Customers
 GROUP BY Customer_ID HAVING COUNT(*) > 1;
SELECT Email, COUNT(*) AS Copies, GROUP_CONCAT(Customer_ID) AS IDs FROM Customers
 WHERE Email IS NOT NULL AND TRIM(Email) <> ''
 GROUP BY LOWER(TRIM(Email)) HAVING COUNT(*) > 1;
```

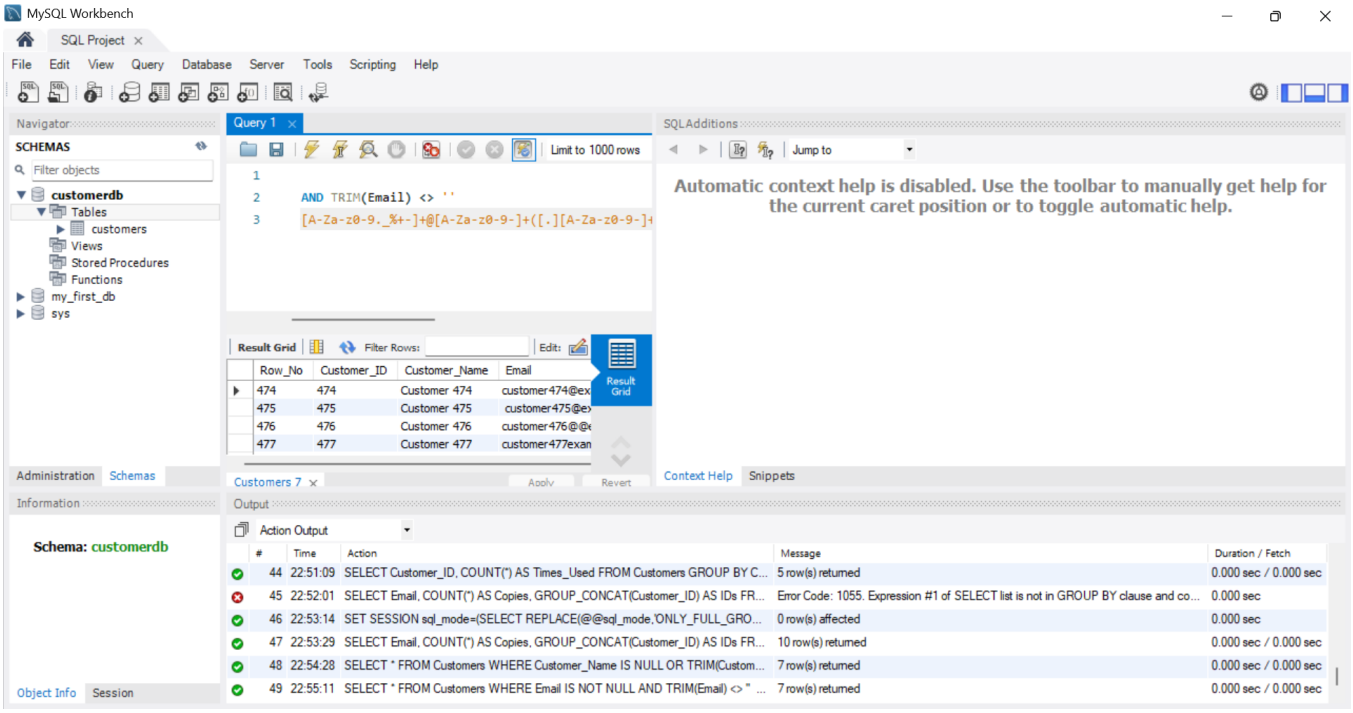
Step 6.3: Missing names



7 rows have a NULL or empty Customer_Name.

```
SELECT * FROM Customers WHERE Customer_Name IS NULL OR TRIM(Customer_Name) = '';
```

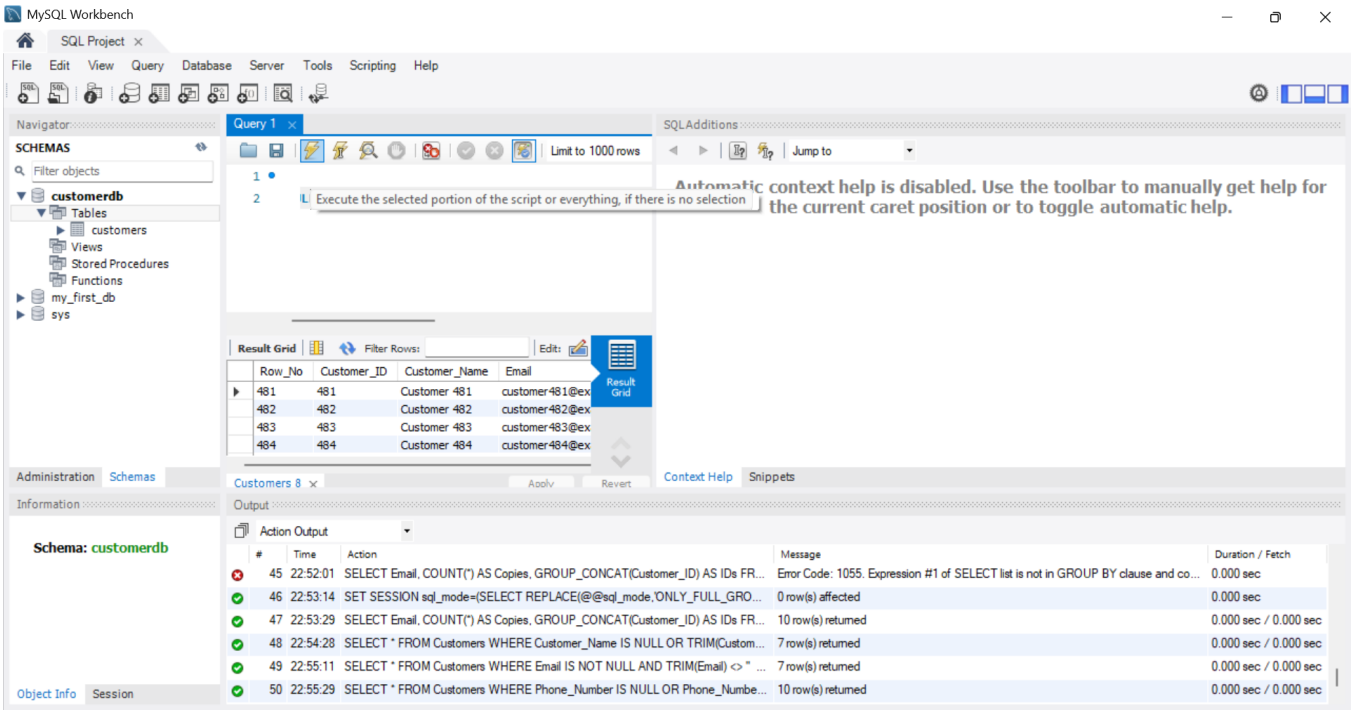
Step 6.5: Invalid email formats



7 emails fail the format check (trailing or leading space, double @, no @, missing .com, empty domain, space inside).

```
SELECT * FROM Customers WHERE Email IS NOT NULL AND TRIM(Email) <> ''
 AND Email NOT REGEXP '^[A-Za-z0-9._%+-]+@[A-Za-z0-9-]+([.][A-Za-z0-9-]+)*[.][A-Za-z]{2,}$';
```

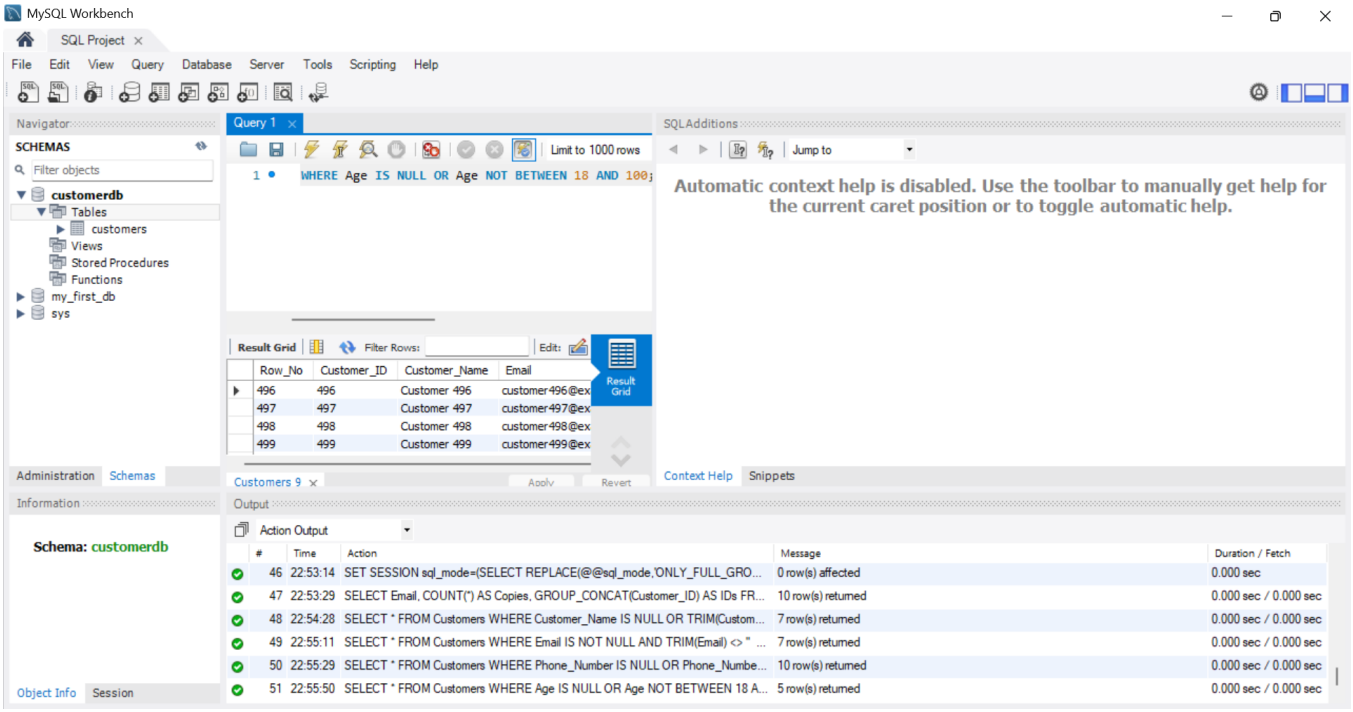
Step 6.6: Incorrect phone numbers



10 phone numbers are not 10 digits starting with 6-9.

```
SELECT * FROM Customers
 WHERE Phone_Number IS NULL OR Phone_Number NOT REGEXP '^[6-9][0-9]{9}$';
```

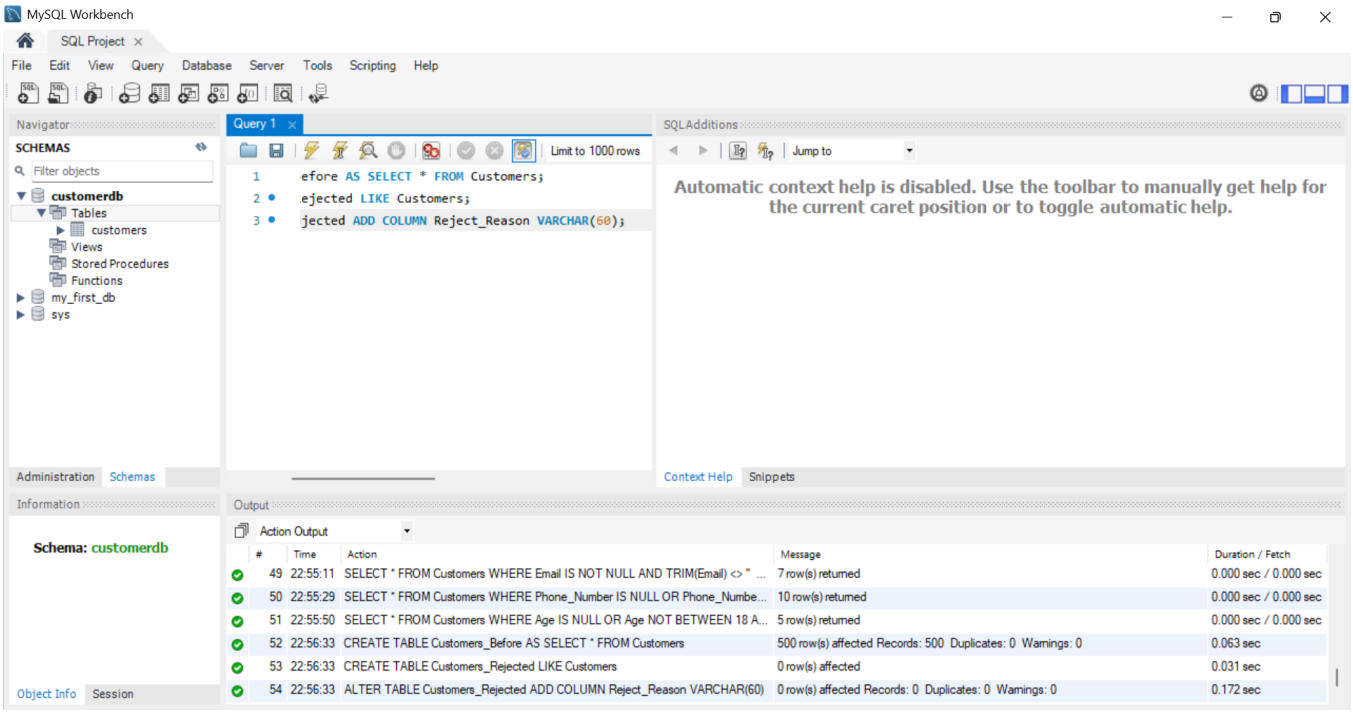
Step 6.7: Invalid ages



5 rows have an age that is NULL or outside 18-100 (negative, 0, 150, 200, NULL).

```
SELECT * FROM Customers WHERE Age IS NULL OR Age NOT BETWEEN 18 AND 100;
```

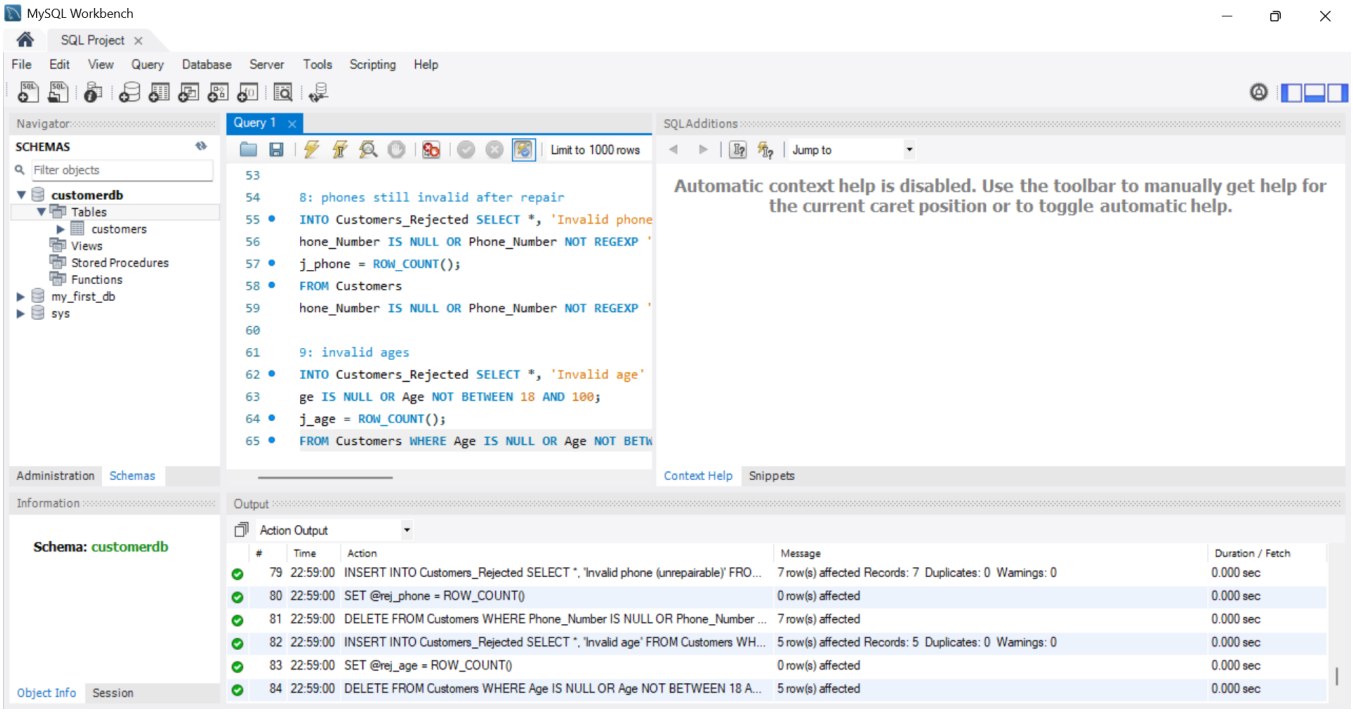
Step 7: Backup and rejected tables



Customers_Before was created with all 500 original rows. Customers_Rejected was created with an extra Reject_Reason column.

```
CREATE TABLE Customers_Before AS SELECT * FROM Customers;
CREATE TABLE Customers_Rejected LIKE Customers;
ALTER TABLE Customers_Rejected ADD COLUMN Reject_Reason VARCHAR(60);
```

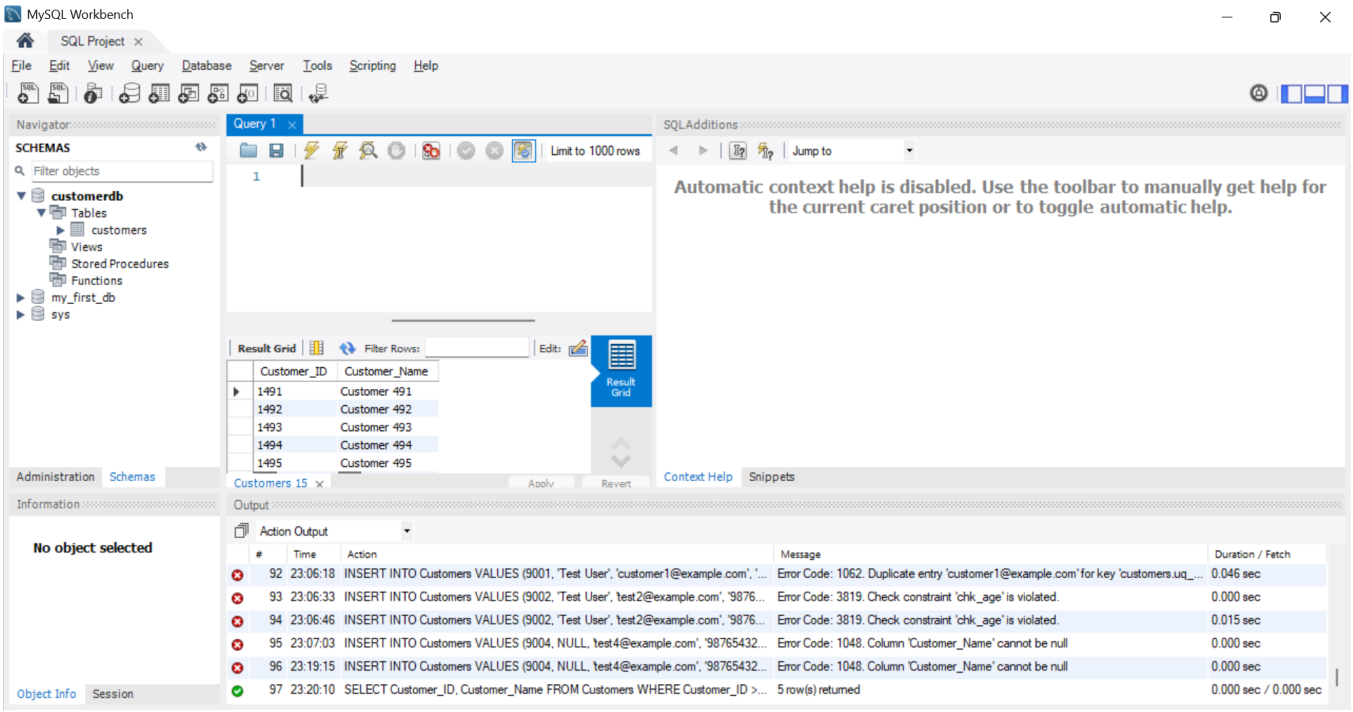
Step 8: Fixes (FIX 8 and FIX 9 shown)



Unrepairable phones (7 rows) and invalid ages (5 rows) were copied to Customers_Rejected and then deleted. Each step uses INSERT into the rejected table first, then DELETE.

```
INSERT INTO Customers_Rejected SELECT *, 'Invalid phone (unrepairable)' FROM Customers
 WHERE Phone_Number IS NULL OR Phone_Number NOT REGEXP '^[6-9][0-9]{9}$';
SET @rej_phone = ROW_COUNT();
DELETE FROM Customers WHERE Phone_Number IS NULL OR Phone_Number NOT REGEXP '^[6-9][0-9]{9}$';
```

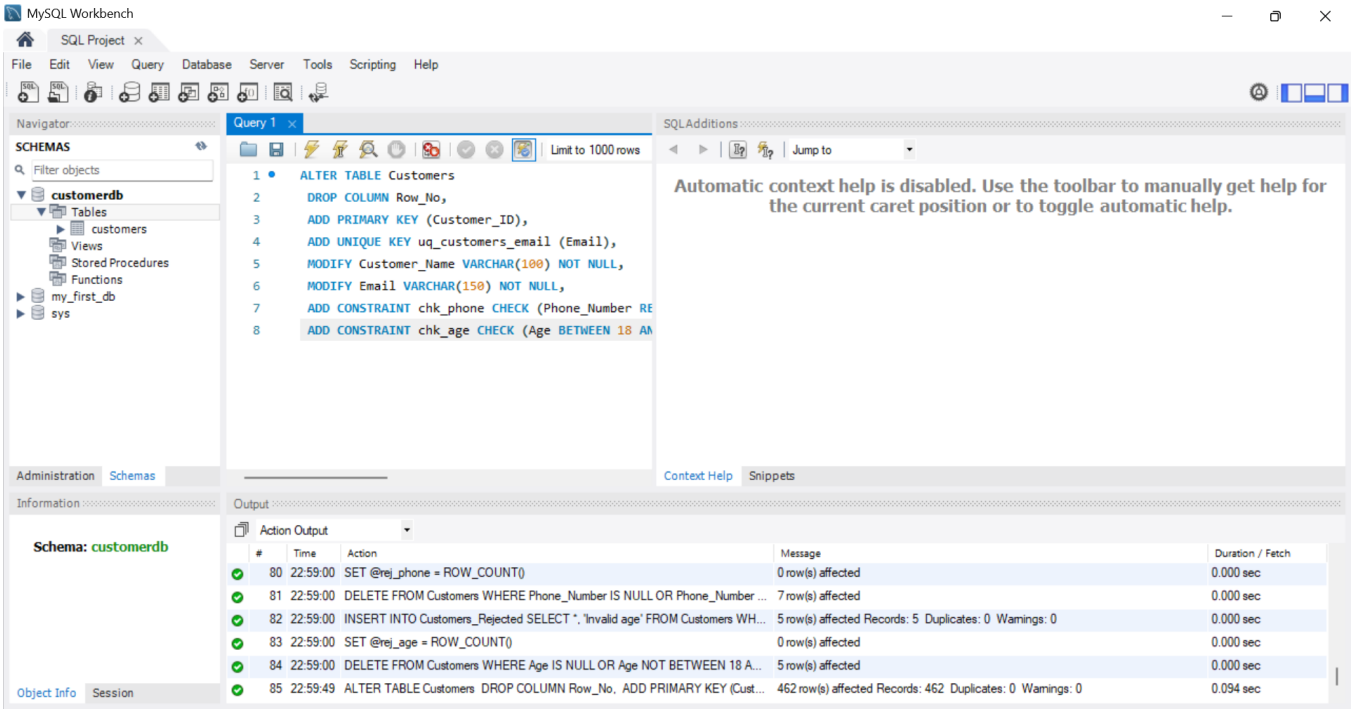
Step 8: Proof that duplicate IDs were repaired



The 5 rows that had a duplicate ID now carry new IDs 1491 to 1495 and are still in the table.

```
SELECT Customer_ID, Customer_Name FROM Customers WHERE Customer_ID > 1000;
```

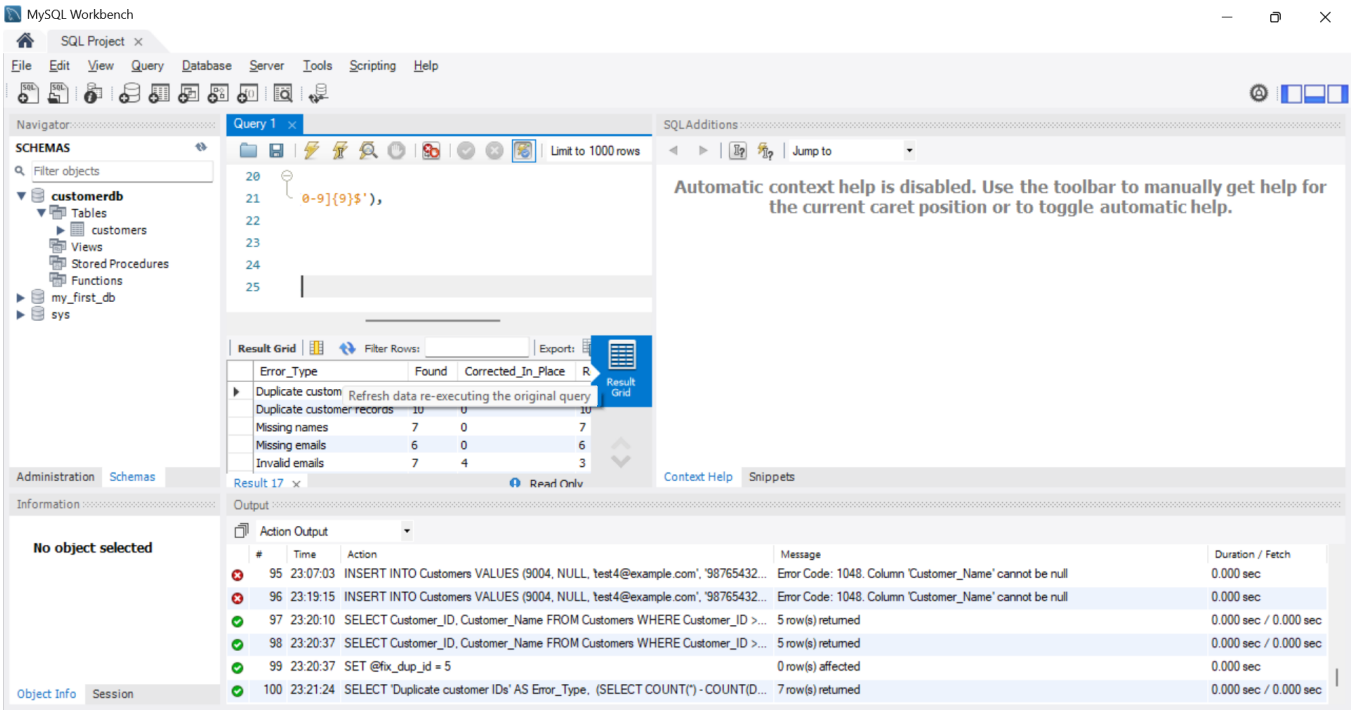
Step 9: Add constraints



ALTER TABLE dropped the helper column Row_No and added the primary key, unique email, NOT NULL, and the two CHECK constraints. 462 rows were carried over without any violation.

```
ALTER TABLE Customers DROP COLUMN Row_No,
 ADD PRIMARY KEY (Customer_ID),
 ADD UNIQUE KEY uq_customers_email (Email),
 MODIFY Customer_Name VARCHAR(100) NOT NULL,
 MODIFY Email VARCHAR(150) NOT NULL,
 ADD CONSTRAINT chk_phone CHECK (Phone_Number REGEXP '^[6-9][0-9]{9}$'),
 ADD CONSTRAINT chk_age CHECK (Age BETWEEN 18 AND 100);
```

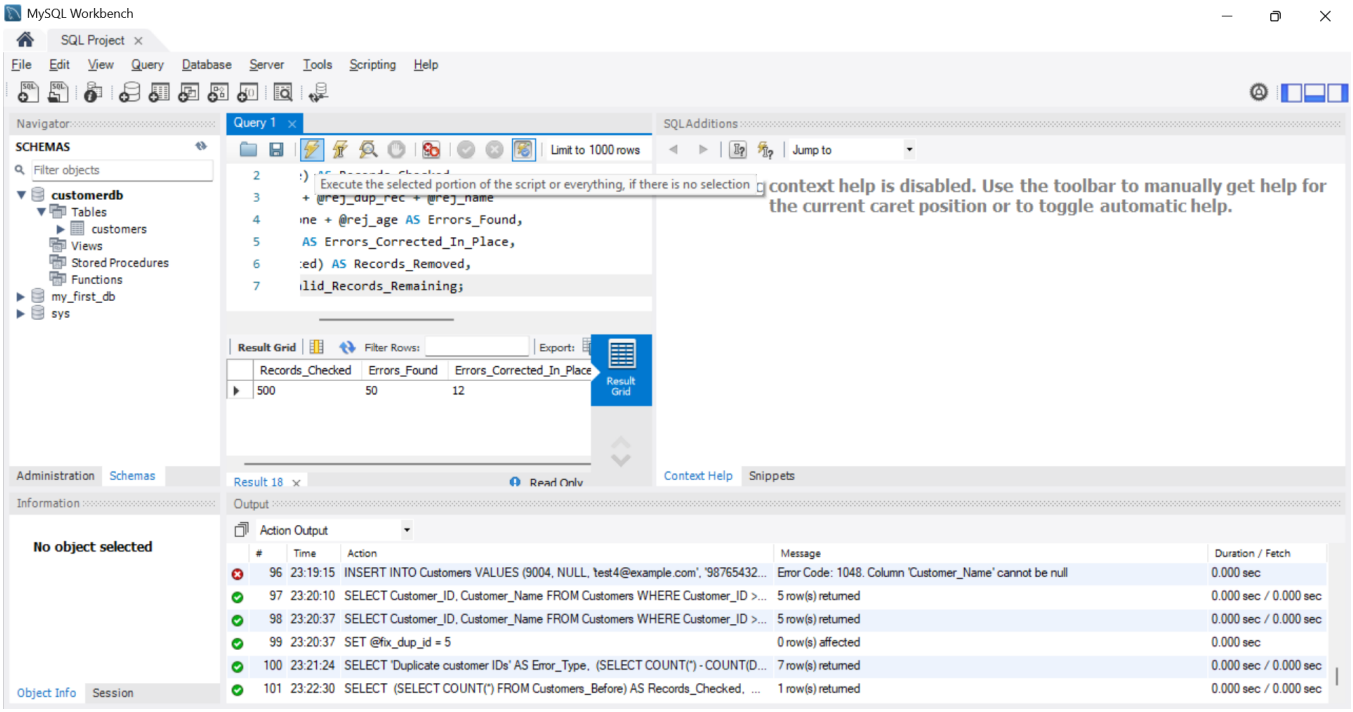
Step 10.1: Error report by category



Errors found, corrected in place and removed for each error type (re-run after correcting the duplicate-ID counter).

```
SELECT 'Duplicate customer IDs' AS Error_Type, (...) AS Found, @fix_dup_id AS Corrected_In_Place, 0 AS Removed
UNION ALL SELECT 'Duplicate customer records', ... UNION ALL ... 'Invalid ages', ...;
```

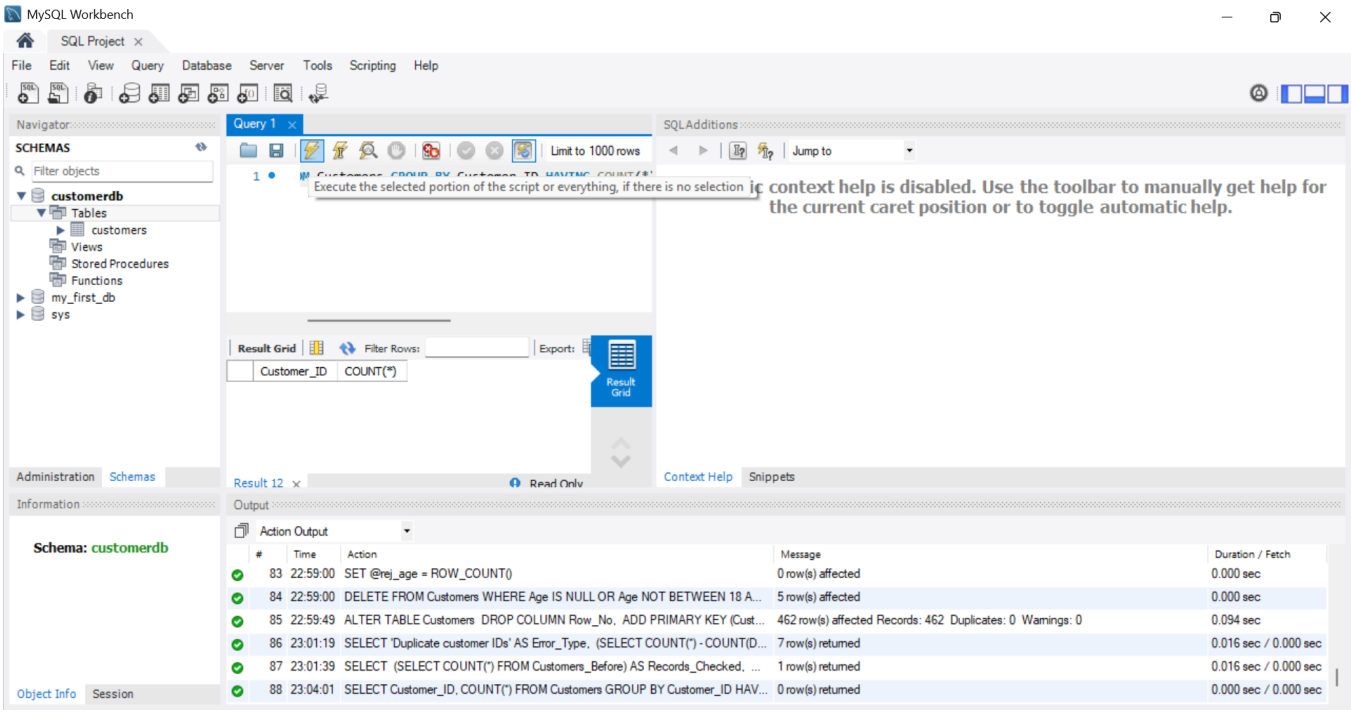
Step 10.2: Before vs after summary



Records checked 500, errors found 50, corrected in place 12 (remaining columns: removed 38, valid remaining 462).

```
SELECT (SELECT COUNT(*) FROM Customers_Before) AS Records_Checked, (...) AS Errors_Found,
 (...) AS Errors_Corrected_In_Place, (SELECT COUNT(*) FROM Customers_Rejected) AS Records_Removed,
 (SELECT COUNT(*) FROM Customers) AS Valid_Records_Remaining;
```

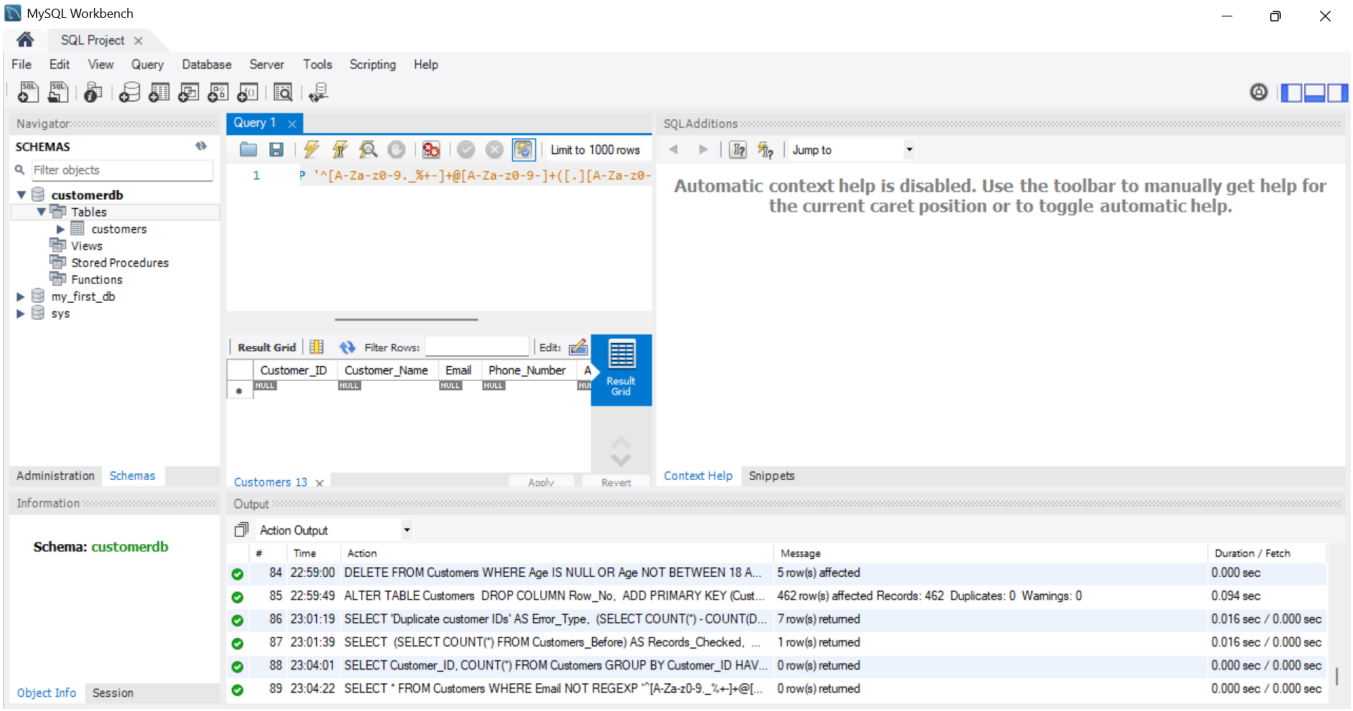
Step 10: Clean check, duplicate IDs



0 rows: no duplicate Customer_ID remains.

```
SELECT Customer_ID, COUNT(*) FROM Customers GROUP BY Customer_ID HAVING COUNT(*) > 1;
```

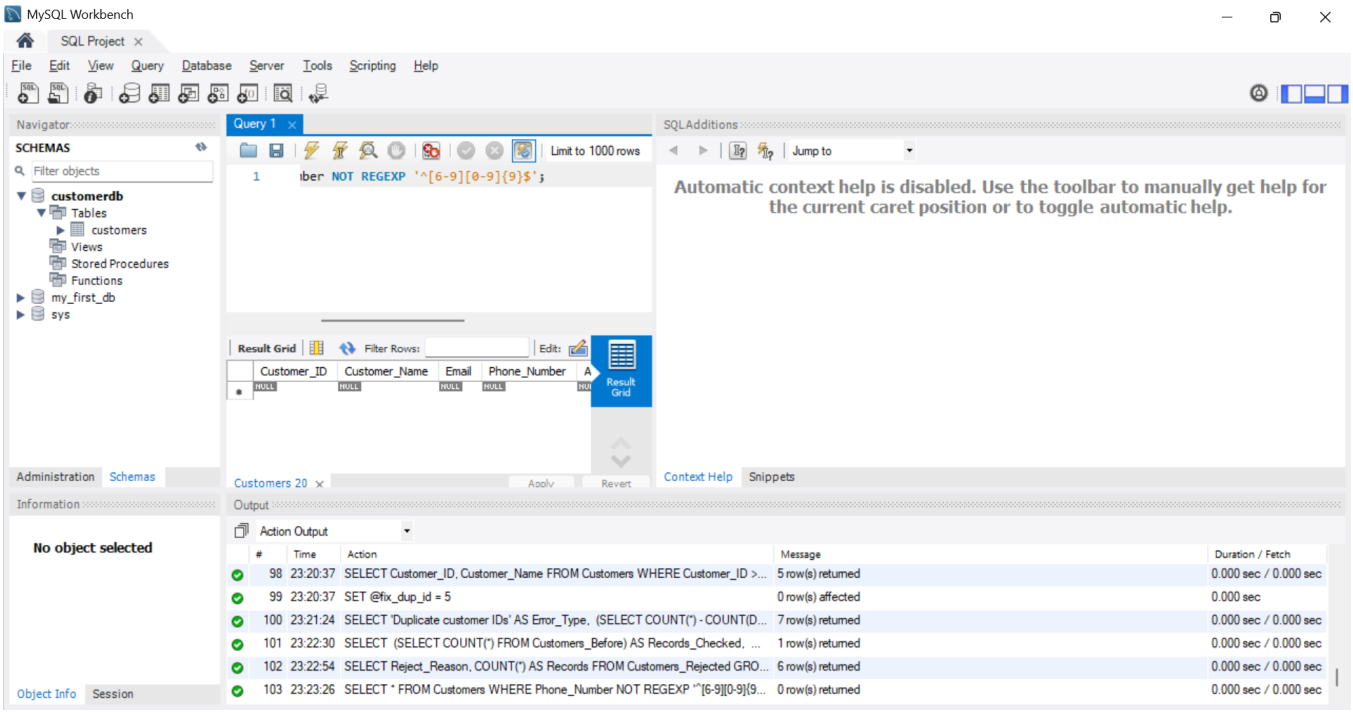
Step 10: Clean check, email format



0 rows: every remaining email matches the format.

```
SELECT * FROM Customers WHERE Email NOT REGEXP
 '^[A-Za-z0-9._%+-]+@[A-Za-z0-9-]+([.][A-Za-z0-9-]+)*[.][A-Za-z]{2,}$';
```

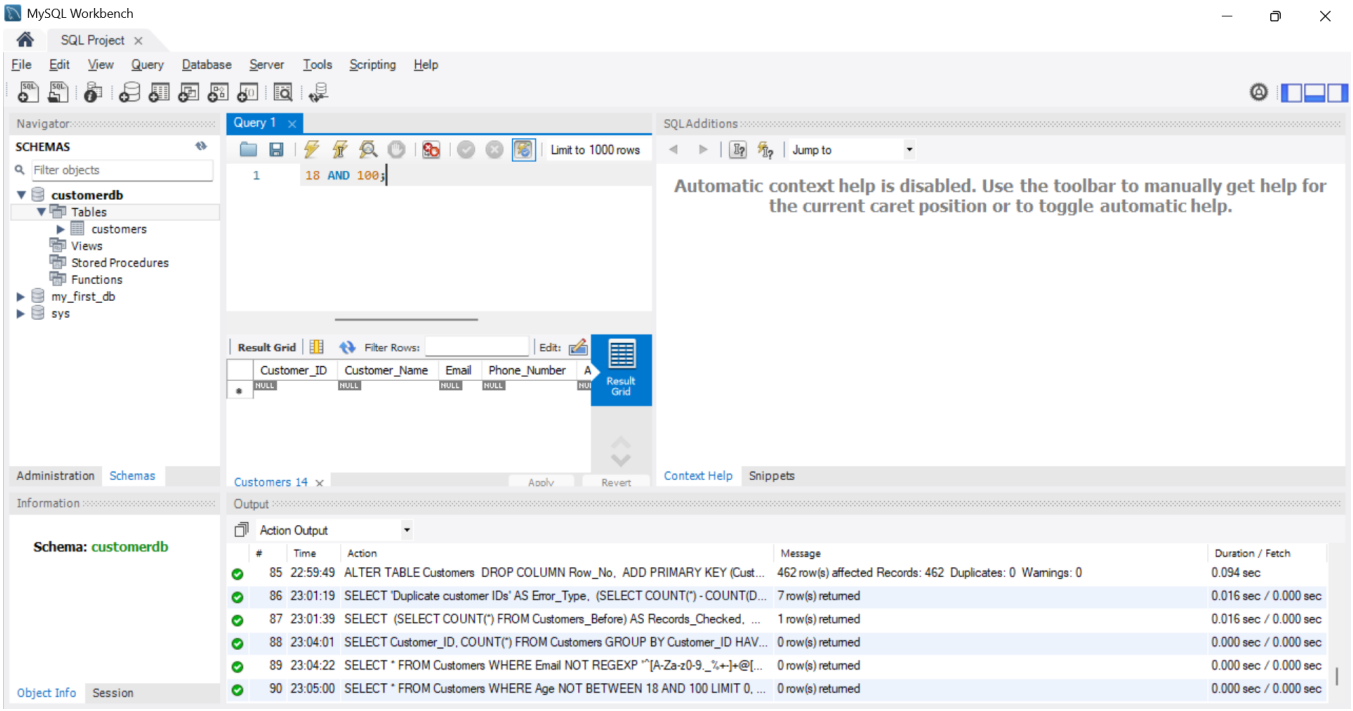
Step 10: Clean check, phone numbers



0 rows: every remaining phone number is 10 digits starting with 6-9. (The Action Output also shows the Reject_Reason summary query, which returned 6 reason groups.)

```
SELECT Reject_Reason, COUNT(*) AS Records FROM Customers_Rejected GROUP BY Reject_Reason;
SELECT * FROM Customers WHERE Phone_Number NOT REGEXP '^[6-9][0-9]{9}$';
```

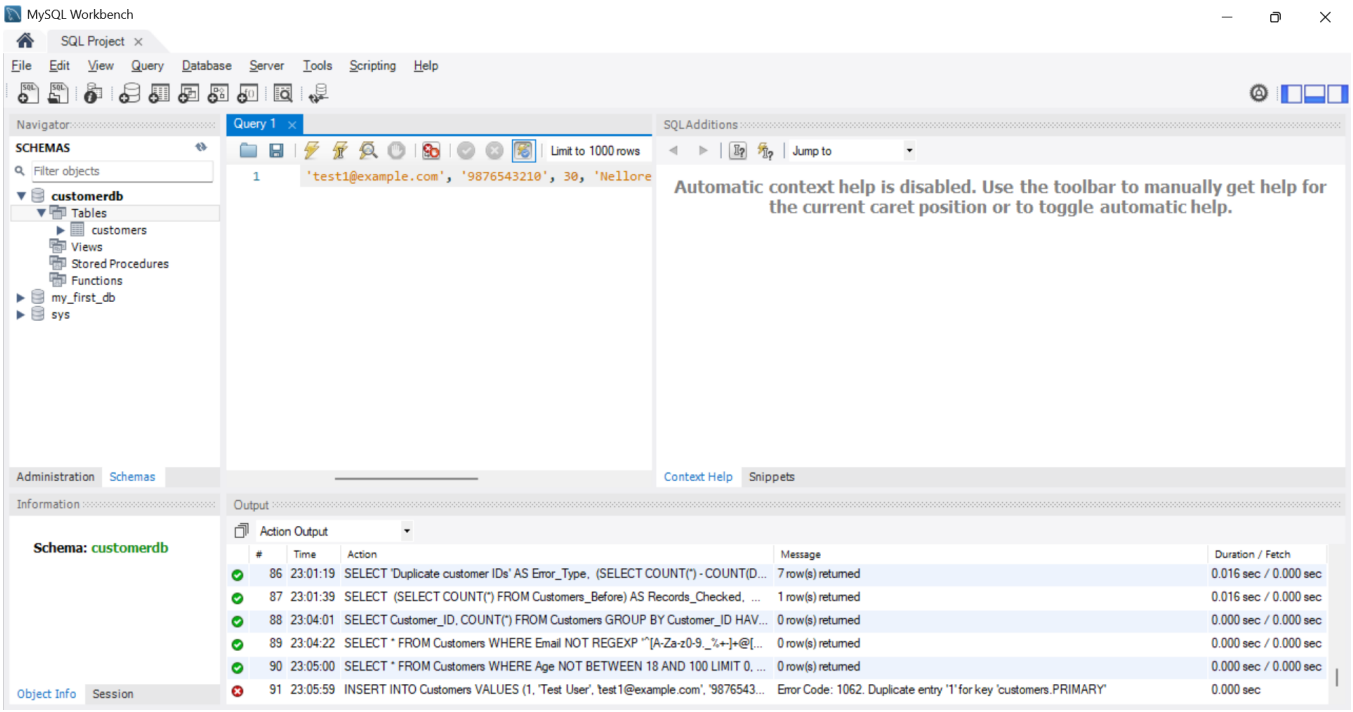
Step 10: Clean check, ages



0 rows: every remaining age is between 18 and 100.

```
SELECT * FROM Customers WHERE Age NOT BETWEEN 18 AND 100;
```

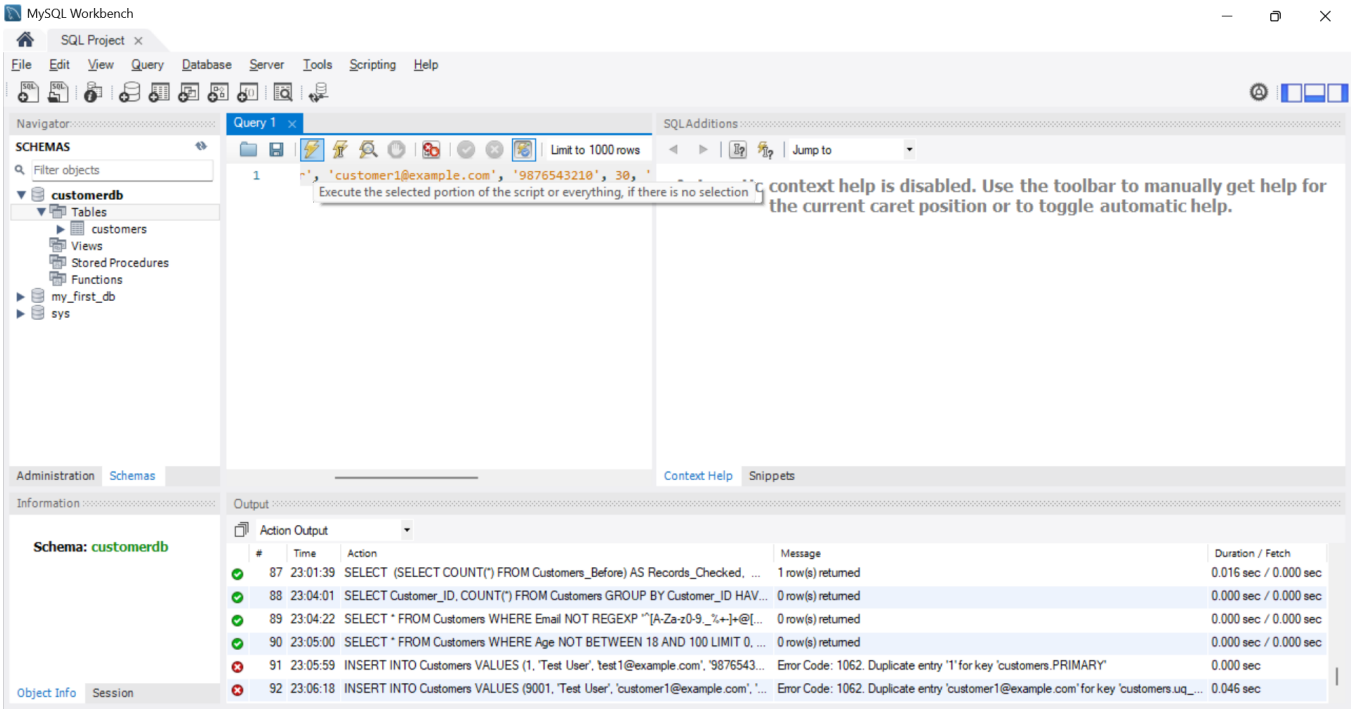
Step 11: Duplicate Customer_ID blocked



Error 1062: duplicate entry for key customers.PRIMARY.

```
INSERT INTO Customers VALUES (1, 'Test User', 'test1@example.com', '9876543210',
 30, 'Nellore', '2025-01-01');
```

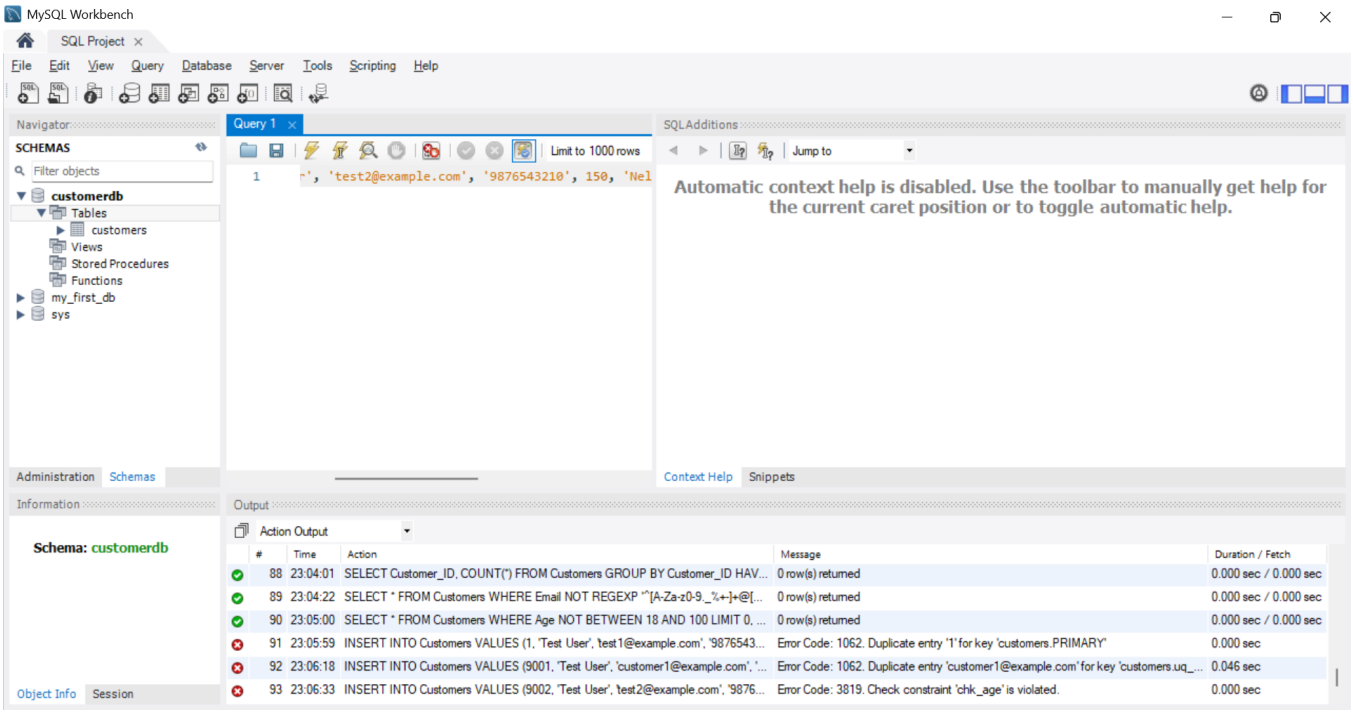
Step 11: Duplicate email blocked



Error 1062: duplicate entry for key customers.uq_customers_email.

```
INSERT INTO Customers VALUES (9001, 'Test User', 'customer1@example.com', '9876543210',
 30, 'Nellore', '2025-01-01');
```

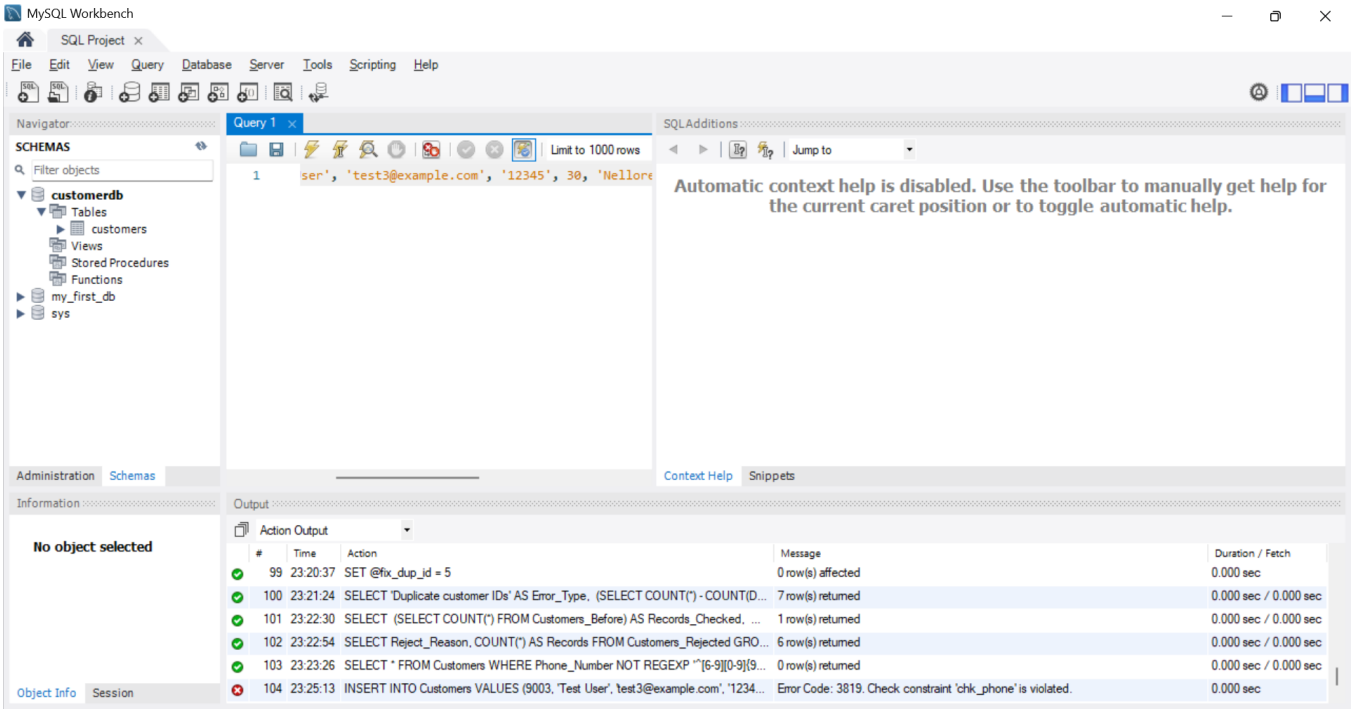
Step 11: Invalid age blocked



Error 3819: check constraint chk_age is violated.

```
INSERT INTO Customers VALUES (9002, 'Test User', 'test2@example.com', '9876543210',
 150, 'Nellore', '2025-01-01');
```

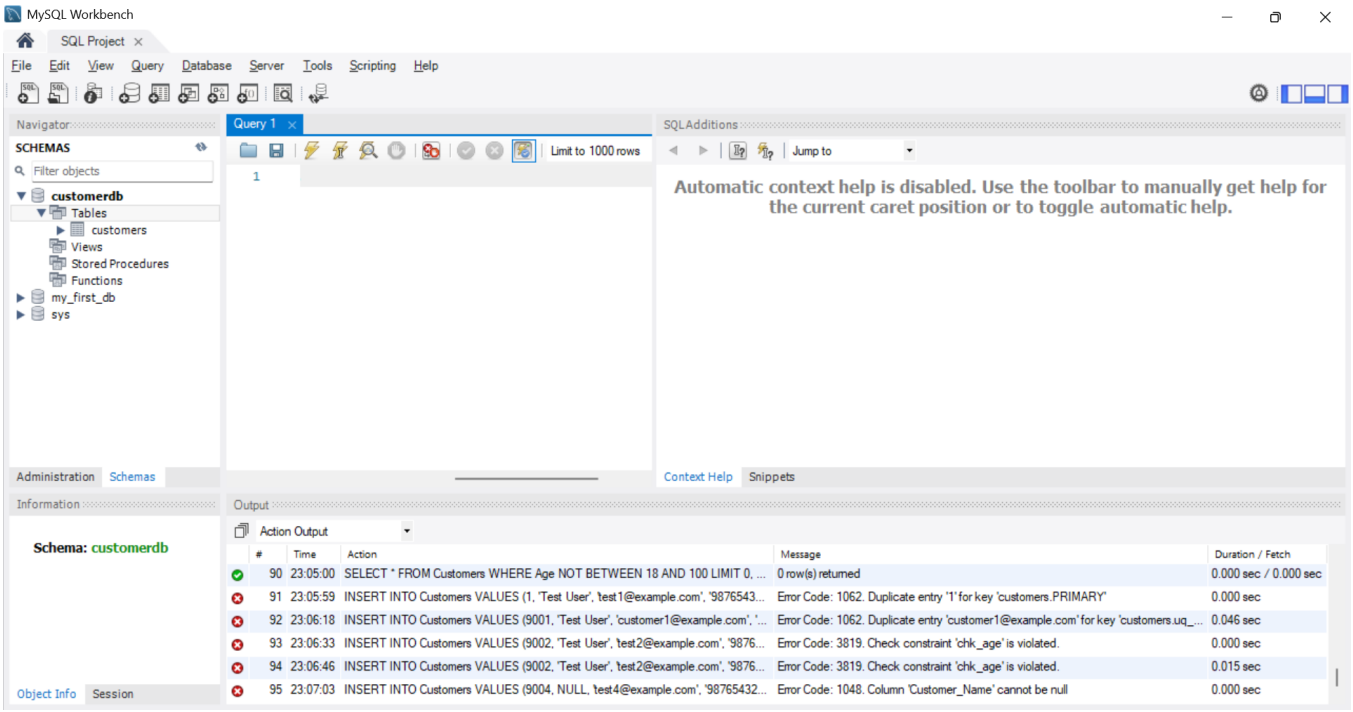
Step 11: Invalid phone blocked



Error 3819: check constraint chk_phone is violated.

```
INSERT INTO Customers VALUES (9003, 'Test User', 'test3@example.com', '12345',
 30, 'Nellore', '2025-01-01');
```

Step 11: Missing name blocked



Error 1048: column Customer_Name cannot be null.

```
INSERT INTO Customers VALUES (9004, NULL, 'test4@example.com', '9876543210',
 30, 'Nellore', '2025-01-01');
```

8. Conclusion

The Customers table started with 500 raw records containing 50 errors across seven categories. Errors were detected with SQL queries, 12 were repaired in place, and 38 records that could not be repaired reliably were moved to Customers_Rejected before being removed. 462 valid records remain. The primary key, unique, NOT NULL and CHECK constraints now stop the same errors from being inserted again, as the Step 11 error messages show.

Appendix A: Complete SQL script

All queries in the order they were run (also submitted as CustomerDB_full.sql).

```
-- ============================================================
-- CustomerDB - Customer Database Management and Error Detection
-- Run in ONE session (the @variables are used in Step 10).
-- Run Step 11 statements one at a time (each must fail).
-- ============================================================
SET SQL_SAFE_UPDATES = 0;   -- avoids Workbench error 1175 on non-key UPDATE/DELETE

-- STEP 1: Create database
DROP DATABASE IF EXISTS CustomerDB;
CREATE DATABASE CustomerDB;
USE CustomerDB;

-- STEP 2: Raw-load table (no constraints on purpose)
CREATE TABLE Customers (
 Row_No INT AUTO_INCREMENT PRIMARY KEY,
 Customer_ID INT,
 Customer_Name VARCHAR(100),
 Email VARCHAR(150),
 Phone_Number VARCHAR(20),
 Age INT,
 City VARCHAR(50),
 Registration_Date DATE
);

-- STEP 3: Insert 500 synthetic records
SET SESSION cte_max_recursion_depth = 1000;
INSERT INTO Customers
 (Row_No, Customer_ID, Customer_Name, Email, Phone_Number, Age, City, Registration_Date)
WITH RECURSIVE n AS (
 SELECT 1 AS i UNION ALL SELECT i + 1 FROM n WHERE i < 500
)
SELECT i, i,
 CONCAT('Customer ', LPAD(i, 3, '0')),
 CONCAT('customer', i, '@example.com'),
 CONCAT(ELT(1 + MOD(i, 4), '6', '7', '8', '9'),
 LPAD(MOD(i * 7919123, 1000000000), 9, '0')),
 18 + MOD(i * 7, 50),
 ELT(1 + MOD(i, 6), 'Nellore','Hyderabad','Chennai','Bengaluru','Vijayawada','Tirupati'),
 DATE_ADD('2023-01-01', INTERVAL MOD(i * 13, 1000) DAY)
FROM n;

SELECT COUNT(*) AS Total_Records FROM Customers;   -- must be 500

-- STEP 4: Inject 50 errors (run in this order)
-- 10 duplicate customer records
UPDATE Customers c JOIN Customers s ON s.Row_No = c.Row_No - 450
SET c.Customer_Name = s.Customer_Name, c.Email = s.Email,
 c.Phone_Number = s.Phone_Number, c.Age = s.Age, c.City = s.City
WHERE c.Row_No BETWEEN 451 AND 460;
-- 7 missing names
UPDATE Customers SET Customer_Name = NULL WHERE Row_No BETWEEN 461 AND 465;
UPDATE Customers SET Customer_Name = '' WHERE Row_No IN (466, 467);
-- 6 missing emails
UPDATE Customers SET Email = NULL WHERE Row_No BETWEEN 468 AND 472;
UPDATE Customers SET Email = '' WHERE Row_No = 473;
-- 7 invalid emails
```

```
UPDATE Customers SET Email = CONCAT(Email, ' ') WHERE Row_No = 474;
UPDATE Customers SET Email = CONCAT(' ', Email) WHERE Row_No = 475;
UPDATE Customers SET Email = REPLACE(Email, '@', '@@') WHERE Row_No = 476;
UPDATE Customers SET Email = REPLACE(Email, '@', '') WHERE Row_No = 477;
UPDATE Customers SET Email = REPLACE(Email, '.com', '') WHERE Row_No = 478;
UPDATE Customers SET Email = 'customer479@.com' WHERE Row_No = 479;
UPDATE Customers SET Email = 'customer 480@example.com' WHERE Row_No = 480;
-- 10 incorrect phone numbers
UPDATE Customers SET Phone_Number = CONCAT('+91', Phone_Number) WHERE Row_No = 481;
UPDATE Customers SET Phone_Number = CONCAT(LEFT(Phone_Number,5),'-',RIGHT(Phone_Number,5)) WHERE
    Row_No = 482;
UPDATE Customers SET Phone_Number = CONCAT(LEFT(Phone_Number,2),' ',SUBSTRING(Phone_Number,3))
    WHERE Row_No = 483;
UPDATE Customers SET Phone_Number = '98765' WHERE Row_No = 484;
UPDATE Customers SET Phone_Number = '12345' WHERE Row_No = 485;
UPDATE Customers SET Phone_Number = 'ABCDE12345' WHERE Row_No = 486;
UPDATE Customers SET Phone_Number = CONCAT(Phone_Number, '1') WHERE Row_No = 487;
UPDATE Customers SET Phone_Number = '1234567890' WHERE Row_No = 488;
UPDATE Customers SET Phone_Number = '0000000000' WHERE Row_No = 489;
UPDATE Customers SET Phone_Number = '5123456789' WHERE Row_No = 490;
-- 5 invalid ages
UPDATE Customers SET Age = -5 WHERE Row_No = 496;
UPDATE Customers SET Age = 0 WHERE Row_No = 497;
UPDATE Customers SET Age = 150 WHERE Row_No = 498;
UPDATE Customers SET Age = 200 WHERE Row_No = 499;
UPDATE Customers SET Age = NULL WHERE Row_No = 500;
-- 5 duplicate customer IDs (LAST)
UPDATE Customers SET Customer_ID = Customer_ID - 390 WHERE Row_No BETWEEN 491 AND 495;

-- STEP 5: Total record count
SELECT COUNT(*) AS Total_Records FROM Customers;

-- STEP 5A: Search / filter / retrieve queries
SELECT * FROM Customers LIMIT 10;
SELECT * FROM Customers WHERE Customer_ID = 25;
SELECT * FROM Customers WHERE City = 'Nellore';
SELECT * FROM Customers WHERE Customer_Name LIKE 'Customer 01%';
SELECT * FROM Customers WHERE Email LIKE '%@example.com' AND Age BETWEEN 25 AND 35;
SELECT * FROM Customers WHERE Registration_Date >= '2024-01-01' ORDER BY Registration_Date DESC
    LIMIT 20;
SELECT City, COUNT(*) AS Total_Customers, ROUND(AVG(Age),1) AS Avg_Age
FROM Customers GROUP BY City ORDER BY Total_Customers DESC;

-- STEP 6: Detect errors
-- 6.1 Duplicate customer IDs (expect 5)
SELECT Customer_ID, COUNT(*) AS Times_Used FROM Customers
GROUP BY Customer_ID HAVING COUNT(*) > 1;
-- 6.2 Duplicate records, same email (expect 10)
-- (needed on MySQL 8 default mode, else error 1055)
SET SESSION sql_mode = (SELECT REPLACE(@@sql_mode, 'ONLY_FULL_GROUP_BY', ''));
SELECT Email, COUNT(*) AS Copies, GROUP_CONCAT(Customer_ID) AS IDs FROM Customers
WHERE Email IS NOT NULL AND TRIM(Email) <> ''
GROUP BY LOWER(TRIM(Email)) HAVING COUNT(*) > 1;
-- 6.3 Missing names (expect 7)
SELECT * FROM Customers WHERE Customer_Name IS NULL OR TRIM(Customer_Name) = '';
-- 6.4 Missing emails (expect 6)
```

```
SELECT * FROM Customers WHERE Email IS NULL OR TRIM(Email) = '';
-- 6.5 Invalid email formats (expect 7)
SELECT * FROM Customers
WHERE Email IS NOT NULL AND TRIM(Email) <> ''
 AND Email NOT REGEXP '^[A-Za-z0-9._%+-]+@[A-Za-z0-9-]+([.][A-Za-z0-9-]+)*[.][A-Za-z]{2,}$';
-- 6.6 Incorrect phones (expect 10)
SELECT * FROM Customers
WHERE Phone_Number IS NULL OR Phone_Number NOT REGEXP '^[6-9][0-9]{9}$';
-- 6.7 Invalid ages (expect 5)
SELECT * FROM Customers WHERE Age IS NULL OR Age NOT BETWEEN 18 AND 100;

-- STEP 7: Backup + rejected table
CREATE TABLE Customers_Before AS SELECT * FROM Customers;
CREATE TABLE Customers_Rejected LIKE Customers;
ALTER TABLE Customers_Rejected ADD COLUMN Reject_Reason VARCHAR(60);

-- STEP 8: Fix errors
-- FIX 1: duplicate IDs
UPDATE Customers c
JOIN (SELECT Row_No, ROW_NUMBER() OVER (PARTITION BY Customer_ID ORDER BY Row_No) AS rn
 FROM Customers) d ON d.Row_No = c.Row_No
SET c.Customer_ID = 1000 + c.Row_No
WHERE d.rn > 1;
SET @fix_dup_id = ROW_COUNT();
-- FIX 2: repairable emails
UPDATE Customers SET Email = REPLACE(Email, ' ', '') WHERE Email LIKE '% %';
SET @fix_email = ROW_COUNT();
UPDATE Customers SET Email = REPLACE(Email, '@@', '@') WHERE Email LIKE '%@@%';
SET @fix_email = @fix_email + ROW_COUNT();
-- FIX 3: repairable phones
UPDATE Customers
SET Phone_Number = REPLACE(REPLACE(REPLACE(Phone_Number,'+91',''),'-',''),' ','')
WHERE Phone_Number NOT REGEXP '^[6-9][0-9]{9}$'
 AND REPLACE(REPLACE(REPLACE(Phone_Number,'+91',''),'-',''),' ','')
 REGEXP '^[6-9][0-9]{9}$';
SET @fix_phone = ROW_COUNT();
-- FIX 4: duplicate records
INSERT INTO Customers_Rejected
SELECT c.*, 'Duplicate customer record' FROM Customers c
JOIN (SELECT Row_No,
 ROW_NUMBER() OVER (PARTITION BY LOWER(TRIM(Email)) ORDER BY Row_No) AS rn
 FROM Customers WHERE Email IS NOT NULL AND TRIM(Email) <> '') d
 ON d.Row_No = c.Row_No
WHERE d.rn > 1;
SET @rej_dup_rec = ROW_COUNT();
DELETE FROM Customers WHERE Row_No IN
 (SELECT Row_No FROM Customers_Rejected WHERE Reject_Reason = 'Duplicate customer record');
-- FIX 5: missing names
INSERT INTO Customers_Rejected SELECT *, 'Missing name' FROM Customers
WHERE Customer_Name IS NULL OR TRIM(Customer_Name) = '';
SET @rej_name = ROW_COUNT();
DELETE FROM Customers WHERE Customer_Name IS NULL OR TRIM(Customer_Name) = '';
-- FIX 6: missing emails
INSERT INTO Customers_Rejected SELECT *, 'Missing email' FROM Customers
WHERE Email IS NULL OR TRIM(Email) = '';
SET @rej_mail = ROW_COUNT();
```

```
DELETE FROM Customers WHERE Email IS NULL OR TRIM(Email) = '';
-- FIX 7: unrepairable emails
INSERT INTO Customers_Rejected SELECT *, 'Invalid email (unrepairable)' FROM Customers
WHERE Email NOT REGEXP '^[A-Za-z0-9._%+-]+@[A-Za-z0-9-]+([.][A-Za-z0-9-]+)*[.][A-Za-z]{2,}$';
SET @rej_bad_mail = ROW_COUNT();
DELETE FROM Customers WHERE Email NOT REGEXP
 '^[A-Za-z0-9._%+-]+@[A-Za-z0-9-]+([.][A-Za-z0-9-]+)*[.][A-Za-z]{2,}$';
-- FIX 8: unrepairable phones
INSERT INTO Customers_Rejected SELECT *, 'Invalid phone (unrepairable)' FROM Customers
WHERE Phone_Number IS NULL OR Phone_Number NOT REGEXP '^[6-9][0-9]{9}$';
SET @rej_phone = ROW_COUNT();
DELETE FROM Customers
WHERE Phone_Number IS NULL OR Phone_Number NOT REGEXP '^[6-9][0-9]{9}$';
-- FIX 9: invalid ages
INSERT INTO Customers_Rejected SELECT *, 'Invalid age' FROM Customers
WHERE Age IS NULL OR Age NOT BETWEEN 18 AND 100;
SET @rej_age = ROW_COUNT();
DELETE FROM Customers WHERE Age IS NULL OR Age NOT BETWEEN 18 AND 100;

-- STEP 9: Constraints
ALTER TABLE Customers
 DROP COLUMN Row_No,
 ADD PRIMARY KEY (Customer_ID),
 ADD UNIQUE KEY uq_customers_email (Email),
 MODIFY Customer_Name VARCHAR(100) NOT NULL,
 MODIFY Email VARCHAR(150) NOT NULL,
 ADD CONSTRAINT chk_phone CHECK (Phone_Number REGEXP '^[6-9][0-9]{9}$'),
 ADD CONSTRAINT chk_age CHECK (Age BETWEEN 18 AND 100);

-- STEP 10: Final report
-- 10.1 Error report by category
SELECT 'Duplicate customer IDs' AS Error_Type,
 (SELECT COUNT(*) - COUNT(DISTINCT Customer_ID) FROM Customers_Before) AS Found,
 @fix_dup_id AS Corrected_In_Place, 0 AS Removed
UNION ALL SELECT 'Duplicate customer records',
 (SELECT COUNT(*) FROM (SELECT ROW_NUMBER() OVER
 (PARTITION BY LOWER(TRIM(Email)) ORDER BY Row_No) rn FROM Customers_Before
 WHERE Email IS NOT NULL AND TRIM(Email) <> '') t WHERE rn > 1), 0, @rej_dup_rec
UNION ALL SELECT 'Missing names',
 (SELECT COUNT(*) FROM Customers_Before
 WHERE Customer_Name IS NULL OR TRIM(Customer_Name) = ''), 0, @rej_name
UNION ALL SELECT 'Missing emails',
 (SELECT COUNT(*) FROM Customers_Before
 WHERE Email IS NULL OR TRIM(Email) = ''), 0, @rej_mail
UNION ALL SELECT 'Invalid emails',
 (SELECT COUNT(*) FROM Customers_Before
 WHERE Email IS NOT NULL AND TRIM(Email) <> '' AND Email NOT REGEXP
 '^[A-Za-z0-9._%+-]+@[A-Za-z0-9-]+([.][A-Za-z0-9-]+)*[.][A-Za-z]{2,}$'),
 @fix_email, @rej_bad_mail
UNION ALL SELECT 'Incorrect phone numbers',
 (SELECT COUNT(*) FROM Customers_Before
 WHERE Phone_Number IS NULL OR Phone_Number NOT REGEXP '^[6-9][0-9]{9}$'),
 @fix_phone, @rej_phone
UNION ALL SELECT 'Invalid ages',
 (SELECT COUNT(*) FROM Customers_Before
 WHERE Age IS NULL OR Age NOT BETWEEN 18 AND 100), 0, @rej_age;
```

```
-- 10.2 Before vs after (expect 500 | 50 | 12 | 38 | 462)
SELECT
 (SELECT COUNT(*) FROM Customers_Before) AS Records_Checked,
 @fix_dup_id + @fix_email + @fix_phone + @rej_dup_rec + @rej_name
 + @rej_mail + @rej_bad_mail + @rej_phone + @rej_age AS Errors_Found,
 @fix_dup_id + @fix_email + @fix_phone AS Errors_Corrected_In_Place,
 (SELECT COUNT(*) FROM Customers_Rejected) AS Records_Removed,
 (SELECT COUNT(*) FROM Customers) AS Valid_Records_Remaining;

-- 10.3 What was removed and why
SELECT Reject_Reason, COUNT(*) AS Records FROM Customers_Rejected GROUP BY Reject_Reason;

-- Clean-check: all four must return 0 rows
SELECT Customer_ID, COUNT(*) FROM Customers GROUP BY Customer_ID HAVING COUNT(*) > 1;
SELECT * FROM Customers WHERE Email NOT REGEXP
 '^[A-Za-z0-9._%+-]+@[A-Za-z0-9-]+([.][A-Za-z0-9-]+)*[.][A-Za-z]{2,}$';
SELECT * FROM Customers WHERE Phone_Number NOT REGEXP '^[6-9][0-9]{9}$';
SELECT * FROM Customers WHERE Age NOT BETWEEN 18 AND 100;

-- STEP 11: Constraints must block bad data (run ONE AT A TIME)
-- Duplicate Customer_ID -> ERROR 1062
INSERT INTO Customers VALUES (1, 'Test User', 'test1@example.com', '9876543210', 30, 'Nellore',
    '2025-01-01');
-- Duplicate email -> ERROR 1062
INSERT INTO Customers VALUES (9001, 'Test User', 'customer1@example.com', '9876543210', 30,
    'Nellore', '2025-01-01');
-- Invalid age -> ERROR 3819
INSERT INTO Customers VALUES (9002, 'Test User', 'test2@example.com', '9876543210', 150,
    'Nellore', '2025-01-01');
-- Invalid phone -> ERROR 3819
INSERT INTO Customers VALUES (9003, 'Test User', 'test3@example.com', '12345', 30, 'Nellore',
    '2025-01-01');
-- Missing name -> ERROR 1048
INSERT INTO Customers VALUES (9004, NULL, 'test4@example.com', '9876543210', 30, 'Nellore',
    '2025-01-01');
```

Appendix B: Questions and answers for the demo

Q1. Why does a PRIMARY KEY stop duplicate IDs, but names, emails and phones still need detection queries?

A PRIMARY KEY makes Customer_ID unique and not null, so the same ID cannot be inserted twice. It says nothing about other columns, so the same person can appear under two different IDs with the same email or phone. Those duplicates are found with GROUP BY ... HAVING COUNT(*) > 1, and email is also protected by a UNIQUE key.

Q2. Why were some records repaired and others removed?

A record is repaired only when the correct value is certain, for example removing a space from an email or +91 from a phone number. If the real value is unknown (missing name, age 200, phone 12345), guessing would create new wrong data, so those rows are moved to Customers_Rejected and deleted from Customers.

Q3. What does the email regular expression check?

It checks for allowed characters before one @, then a domain name, then a dot, and then an ending of at least two letters. Emails with no @, a double @, a space, or an empty domain fail.

Q4. What does the phone regular expression check?

^[6-9][0-9]{9}$ means the number is exactly 10 digits and the first digit is 6, 7, 8 or 9, which is the Indian mobile format. Letters, short numbers, an extra digit and numbers starting with 0-5 fail.

Q5. Why were Customers_Before and Customers_Rejected kept?

Customers_Before keeps the original 500 rows as proof for the before/after report. Customers_Rejected keeps every removed row with its reason, so nothing is lost silently and the rows can be reviewed later.

Q6. Why was the Customers table created without constraints at first?

To let the bad data load so that the detection and cleaning steps can be shown. Constraints were added only after cleaning, because ALTER TABLE would fail on bad rows.

Q7. What does ROW_NUMBER() do in the fixes?

It numbers the rows inside each group (for example, each Customer_ID or each email) in order of Row_No. Row number 1 is the original; rows with a number above 1 are the later copies that get a new ID or are moved to the rejected table.

Q8. Why was a counter problem seen in the report, and how was it handled?

The session variable @fix_dup_id held 0 in the first report run, so the totals showed 45 and 7 instead of 50 and 12. I confirmed that 5 rows had new IDs (1491 to 1495), set the variable to 5, and re-ran the report.

Q9. Which constraint produced which error message in Step 11?

Duplicate ID gave error 1062 (primary key), duplicate email gave 1062 (unique key), invalid age gave 3819 (chk_age), invalid phone gave 3819 (chk_phone), and a missing name gave 1048 (NOT NULL).

Appendix C: What each query and fix does

Detection queries (Step 6)

Query	What it does	Result
6.1 Duplicate IDs	Groups rows by Customer_ID and keeps groups with more than one row.	5 IDs
6.2 Duplicate records	Groups rows by lower-case, trimmed email and keeps groups with more than one row. GROUP_CONCAT lists the IDs involved.	10 groups
6.3 Missing names	Selects rows where Customer_Name is NULL or only spaces after TRIM.	7 rows
6.4 Missing emails	Selects rows where Email is NULL or only spaces after TRIM.	6 rows
6.5 Invalid emails	Selects non-empty emails that do not match the email regular expression.	7 rows
6.6 Wrong phones	Selects phones that are NULL or do not match ^[6-9][0-9]{9}$.	10 rows
6.7 Invalid ages	Selects ages that are NULL or not between 18 and 100.	5 rows

Fixes (Step 8)

Fix	What it does	Outcome
FIX 1 Duplicate IDs	Numbers rows inside each Customer_ID with ROW_NUMBER(). Rows numbered above 1 get a new unique ID (1000 + Row_No). The first row keeps its ID.	5 corrected
FIX 2 Emails	REPLACE removes spaces and turns @@ into @ where the email contains them.	4 corrected
FIX 3 Phones	Removes +91, dashes and spaces, but only where the cleaned value becomes a valid 10-digit number starting 6-9.	3 corrected
FIX 4 Duplicate records	Numbers rows inside each email. Rows numbered above 1 are copied to Customers_Rejected with reason 'Duplicate customer record' and deleted.	10 removed
FIX 5 Missing names	Copies rows with a missing name to Customers_Rejected, then deletes them.	7 removed
FIX 6 Missing emails	Copies rows with a missing email to Customers_Rejected, then deletes them.	6 removed
FIX 7 Bad emails	After the repairs, emails still failing the pattern are copied to Customers_Rejected and deleted.	3 removed
FIX 8 Bad phones	After the repairs, phones still failing the pattern are copied to Customers_Rejected and deleted.	7 removed
FIX 9 Bad ages	Rows with an age that is NULL or outside 18-100 are copied to Customers_Rejected and deleted.	5 removed

Order matters: repairs (FIX 1 to 3) run first so that fixable rows are not thrown away, and the removals (FIX 4 to 9) run after. Every removal inserts into Customers_Rejected before the DELETE, and the row count of each INSERT is saved in a variable (ROW_COUNT) for the final report.

Search and filter queries

These queries retrieve customer information from the cleaned table. Run them on Customers.

```
SELECT * FROM Customers WHERE City = 'Nellore';
SELECT * FROM Customers WHERE Age BETWEEN 25 AND 35 ORDER BY Age;
SELECT * FROM Customers WHERE Customer_Name LIKE 'Customer 01%';
SELECT * FROM Customers WHERE Registration_Date >= '2024-01-01'
 ORDER BY Registration_Date DESC LIMIT 20;
SELECT City, COUNT(*) AS Total_Customers, ROUND(AVG(Age),1) AS Avg_Age
 FROM Customers GROUP BY City ORDER BY Total_Customers DESC;
```